React

From Basics to Advanced

The complete guide: concepts, examples, diagrams & tricky points

- How React renders: Virtual DOM, reconciliation, Fiber
- JSX, Components, Props, State & every Hook
- Context, Redux Toolkit, React Query, Router, Performance
- React 19, Server Components, TypeScript, Testing & interview puzzles

Prepared by Kaif Shaikh • Study & Interview Edition

Table of Contents

1. Introduction to React 6

1.1 How to Use This Guide 6
1.2 What is React? 6
1.3 Library vs Framework, and the React Ecosystem 7
1.4 SPA, MPA and SSR 8
1.5 How React Works Under the Hood 9
1.6 Setting Up a Project 10
1.7 Your First React App 11
1.8 React Versions at a Glance 12

2. JavaScript Essentials for React 13

2.1 let, const and var 13
2.2 Arrow Functions 13
2.3 Destructuring 14
2.4 Spread and Rest 14
2.5 Template Literals, Optional Chaining, Nullish Coalescing 14
2.6 Truthy and Falsy 15
2.7 Array Methods You Will Use Daily 15
2.8 Equality and References 16
2.9 Modules: import and export 16
2.10 Closures 16
2.11 Promises and async/await 17
2.12 The Event Loop in One Minute 17

3. JSX in Depth 18

3.1 What is JSX? 18
3.2 JSX Rules 18
3.3 Fragments 19
3.4 Conditional Rendering 19
3.5 Rendering Lists and Keys 20
3.6 Rendering HTML Strings and Escaping 21

4. Components and Props 22

4.1 Function Components 22
4.2 Props 22
4.3 The children Prop and Composition 23
4.4 One-Way Data Flow 24
4.5 Pure Components 24
4.6 Prop Validation: TypeScript and PropTypes 25
4.7 Component Design Tips 25

5. State and useState 26

5.1 What is State? 26

5.2 State is a Snapshot 27

5.3 Batching 27

5.4 Lazy Initial State 28

5.5 Updating Objects Immutably 28

5.6 Updating Arrays Immutably 29

5.7 Lifting State Up 29

5.8 Derived State: Don't Store What You Can Calculate 30

5.9 State Preservation and Position in the Tree 30

6. Events and Forms 31

6.1 Handling Events 31

6.2 Synthetic Events 31

6.3 Controlled Components 32

6.4 Uncontrolled Components 33

6.5 React 19 Form Actions 33

6.6 Form Libraries 34

7. Rendering, Re-rendering and Reconciliation 35

7.1 Trigger, Render, Commit 35

7.2 When Does a Component Re-render? 35

7.3 Reconciliation and the Diffing Algorithm 36

7.4 Mounting, Updating, Unmounting 37

7.5 Class Components and Lifecycle Methods (Legacy) 37

8. useEffect and Side Effects 39

8.1 What is an Effect? 39

8.2 Common Examples 40

8.3 Fetching Data in an Effect (and Race Conditions) 40

8.4 Dependency Array Pitfalls 41

8.5 You Might Not Need an Effect 42

8.6 useLayoutEffect 42

8.7 useEffectEvent (React 19.2) 43

9. Hooks in Depth 44

9.1 Rules of Hooks 44

9.2 Hooks Overview 45

9.3 useRef 45

9.4 useReducer 47

9.5 useContext 47

9.6 useMemo 48

9.7 useCallback 48

9.8 useId 49

9.9 useTransition and useDeferredValue (Concurrent Features) 49
9.10 useSyncExternalStore 50
9.11 React 19 Hooks: use, useOptimistic 50
9.12 Custom Hooks 51
10. Context API and State Management 52
10.1 The Problem: Prop Drilling 52
10.2 Context API Step by Step 53
10.3 Types of State 54
10.4 Redux Toolkit (RTK) 55
10.5 Zustand (Lightweight Alternative) 56
10.6 Choosing a Tool 56
11. Routing with React Router 57
11.1 Client-Side Routing 57
11.2 Basic Setup 57
11.3 Params, Search Params and Navigation 58
11.4 Nested Routes and Layouts 58
11.5 Protected Routes 59
11.6 Data Routers: Loaders and Actions 59
11.7 Lazy-Loaded Routes 59
12. Data Fetching and APIs 61
12.1 Approaches Compared 61
12.2 axios with Interceptors (JWT Example) 61
12.3 TanStack Query (React Query) 62
12.4 Loading, Error and Empty States 62
12.5 Suspense for Data 63
12.6 Avoiding Waterfalls 63
13. Performance Optimization 64
13.1 React.memo 64
13.2 Avoiding Unnecessary Work 65
13.3 Code Splitting and Lazy Loading 65
13.4 Large Lists: Virtualization 65
13.5 Debouncing and Throttling 66
13.6 React Compiler 66
13.7 Other Performance Checklist 66
14. Advanced Patterns 68
14.1 Higher-Order Components (HOC) 68
14.2 Render Props 68
14.3 Compound Components 69
14.4 Controlled vs Uncontrolled Component APIs 69
14.5 Portals 69

14.6 Error Boundaries ... 70
14.7 Suspense and lazy (Recap) ... 70
14.8 Container / Presentational and Custom Hooks ... 71
14.9 Polymorphic and Slot Components ... 71
15. Styling, TypeScript and Testing ... 72
15.1 Styling Options ... 72
15.2 TypeScript with React ... 73
15.3 Testing with React Testing Library ... 73
16. React 18/19, Server Components and Next.js ... 75
16.1 React 18 Recap ... 75
16.2 React 19 Highlights ... 76
16.3 React Server Components (RSC) ... 77
16.4 Hydration ... 78
16.5 Next.js App Router Essentials ... 79
16.6 Accessibility (a11y) Essentials ... 79
16.7 Security Checklist ... 79
17. Tricky Interview Questions and Output Puzzles ... 81
17.1 Output Prediction Puzzles ... 81
17.2 Frequently Asked Conceptual Questions ... 85
17.3 Machine-Coding Questions to Practise ... 87
18. Quick Revision Cheat Sheet ... 88
18.1 Syntax at a Glance ... 88
18.2 Top 25 Tricky Points ... 88
18.3 Suggested Study Roadmap ... 89

1. Introduction to React

1.1 How to Use This Guide

> **✎ Definition**
>
> A study guide that explains each React topic in the same order: a simple explanation, a code example, then the tricky points and interview tips that go with it.

This guide takes you from "what is React?" to advanced topics: hooks internals, reconciliation, performance, state management, React 19 and Server Components. Each topic follows the pattern **simple explanation, then code, then tricky points**. Coloured boxes help you scan:

> **☼ Simple Analogy**
>
> A real-world comparison that makes the idea stick.

> **⚡ Tricky Point**
>
> Tricky points: behaviour that surprises people and is loved by interviewers.

> **★ Interview Tip**
>
> Interview tip: how to answer the question in one or two crisp lines.

> **⚠ Watch Out**
>
> Watch out: a common bug or bad practice.

1.2 What is React?

> **✎ Definition**
>
> **React** is an open-source JavaScript **library** from Meta for building user interfaces out of reusable **components**. It updates the page efficiently when data (state) changes.

React is an open-source **JavaScript library for building user interfaces**, created by Jordan Walke at Facebook (Meta) and released in 2013. It is "the library for web and native user interfaces": React DOM renders to the browser, React Native renders to iOS/Android.

Core ideas:

- **Component-based:** the UI is split into small, reusable pieces (components) that are just JavaScript functions returning UI.
- **Declarative:** you describe *what* the UI should look like for a given state; React figures out *how* to update the DOM.
- **One-way data flow:** data flows from parent to child via props; children talk back via callback functions.
- **UI = f(state):** when state changes, React re-runs your component and updates only what changed.
- **Learn once, write anywhere:** web, mobile (React Native), desktop (Electron), even VR and terminals.

> ☼ **Simple Analogy**
>
> Imperative code (jQuery) is like giving a taxi driver turn-by-turn directions. Declarative code (React) is like telling the driver the destination: "Show this list of 5 transactions." React works out the turns (DOM operations) itself.

Imperative vs declarative

```
// Imperative (vanilla JS): YOU manipulate the DOM step by step
const btn = document.getElementById('btn');
let count = 0;
btn.addEventListener('click', () => {
  count++;
  document.getElementById('label').textContent = `Clicked ${count} times`;
});

// Declarative (React): describe UI for the current state
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

1.3 Library vs Framework, and the React Ecosystem

> ✎ **Definition**
>
> A **library** solves one job and you call it (React handles the UI). A **framework** controls the whole app structure and calls your code (Angular, Next.js). The React **ecosystem** adds routing, state, data-fetching and build tools around it.

React itself only handles the **view layer**. A full app combines it with other tools:

Need	Popular choice
Build tool / dev server	Vite (recommended for SPAs); Create React App is deprecated
Full-stack framework	Next.js, React Router v7 (framework mode, formerly Remix), Expo for mobile
Routing	React Router, TanStack Router
Server state / data fetching	TanStack Query (React Query), SWR, RTK Query
Client state	useState/useReducer/Context, Redux Toolkit, Zustand, Jotai
Forms	React Hook Form + Zod, Formik
Styling	Tailwind CSS, CSS Modules, styled-components, MUI, shadcn/ui
Testing	Vitest/Jest + React Testing Library, Playwright/Cypress
Language	TypeScript (strongly recommended)

React	Angular	Vue
Library (view layer)	Full framework	Progressive framework
JSX (JS + markup)	HTML templates + TypeScript	HTML templates or JSX
One-way data flow	Two-way binding available	Two-way binding with v-model
Virtual DOM	Real DOM + change detection (signals)	Virtual DOM + reactivity system
You choose router/state libs	Batteries included	Official router/store

1.4 SPA, MPA and SSR

> **✎ Definition**
>
> **SPA** (Single-Page App): one HTML page, and JavaScript renders every view in the browser. **MPA**: the server sends a new page for each navigation. **SSR**: the server renders HTML for each request, which the client then makes interactive.



Figure 1.1 - Multi-page app vs single-page app

- **SPA (Single Page Application):** the server sends one HTML page and a JS bundle; React renders everything in the browser and swaps views without reloading.
- **CSR (Client-Side Rendering):** HTML is built in the browser. Simple, but slower first paint and weaker SEO.
- **SSR (Server-Side Rendering):** HTML is built on the server per request, then **hydrated** (React attaches event handlers) in the browser.
- **SSG (Static Site Generation):** HTML built at build time. **ISR** regenerates static pages periodically.
- **RSC (React Server Components):** components that run only on the server and send no JS to the client (Chapter 17).

1.5 How React Works Under the Hood

✎ Definition

React keeps a lightweight **Virtual DOM** description of the UI. On each update it **reconciles** (diffs) the new description with the previous one and applies only the needed changes to the real DOM. The **Fiber** engine schedules this work.

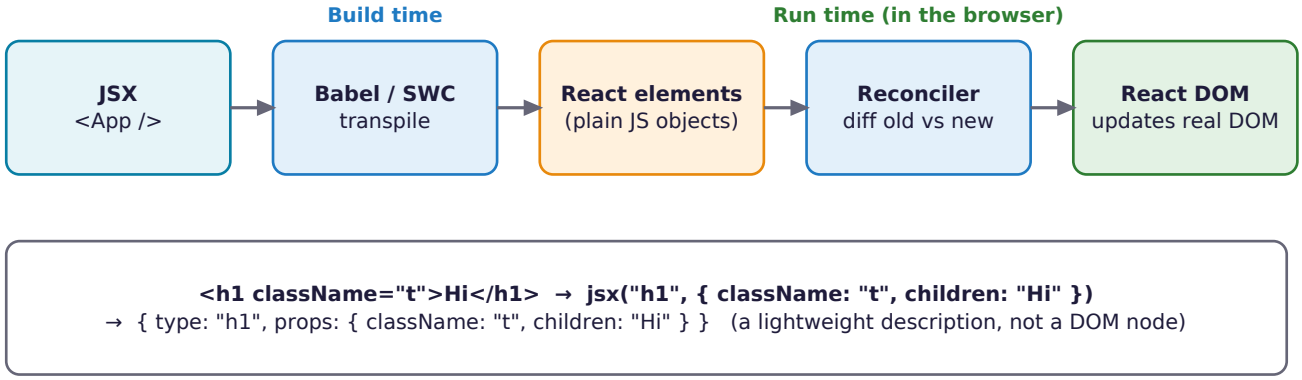


Figure 1.2 - From JSX to the real DOM

1. You write **JSX**. A compiler (Babel, SWC, esbuild) turns it into function calls: `jsx('h1', {...})` (older: `React.createElement`). 2. Those calls produce **React elements**: plain, immutable JavaScript objects describing the UI. 3. React keeps a tree of these descriptions (often called the **Virtual DOM**) plus internal **Fiber** nodes holding state and effects. 4. When state changes, React re-runs the affected components, builds a new tree and **diffs** it with the previous one (**reconciliation**). 5. React DOM applies only the **minimal set of changes** to the real DOM (**commit**).

★ Interview Tip

"What is the Virtual DOM?" - A lightweight in-memory JavaScript representation of the UI. On every update React creates a new one, compares it with the previous one (diffing) and applies only the differences to the real DOM, which is slow to manipulate directly.

⚡ Tricky Point

The Virtual DOM is **not** automatically "faster than the DOM". Hand-written precise DOM updates are faster. Its value is that you write simple declarative code and still get reasonably efficient, batched updates. Frameworks like Svelte and Solid skip the Virtual DOM entirely.

React Fiber

Fiber (React 16+) is the reconciliation engine. Each component instance is a "fiber" node, a unit of work. Fiber lets React **split rendering into chunks, pause, prioritise and resume** it, which powers concurrent features like `useTransition` and Suspense. It also keeps two trees: the **current** tree (on screen) and the **work-in-progress** tree being prepared.

1.6 Setting Up a Project

> ✎ **Definition**
>
> Creating a new React project with a build tool such as **Vite** (or a framework like Next.js), which provides a dev server, JSX compilation, hot reload and production bundling.

```
// Vite + React + TypeScript (recommended for SPAs)
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev          // dev server with hot module replacement (HMR) at localhost:5173
npm run build        // production bundle in /dist
npm run preview      // preview the production build

// Next.js (full-stack framework)
npx create-next-app@latest my-next-app
```

Typical project structure

```
my-app/
  index.html            // single HTML page with <div id="root"></div>
  package.json          // dependencies + scripts
  vite.config.ts
  src/
    main.tsx            // entry point: createRoot(...).render(<App />)
    App.tsx             // root component
    components/         // reusable UI: Button, Modal, Table
    features/           // feature folders: transactions/, auth/, reports/
    hooks/              // custom hooks: useDebounce, useAuth
    services/ or api/   // API clients
    store/              // Redux/Zustand state
    types/              // TypeScript types
```

1.7 Your First React App

✎ Definition

A minimal React application: a root component rendered into a DOM element with `createRoot(...).render(<App />)`.

```jsx
// main.jsx - mounting React into the page
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

// App.jsx - a component is just a function that returns JSX
export default function App() {
  const name = 'Kaif';
  return (
    <main>
      <h1>Hello, {name}!</h1>
      <p>Today is {new Date().toLocaleDateString()}</p>
    </main>
  );
}
```

⚡ Tricky Point

`ReactDOM.render(<App />, el)` (React 17 and earlier) was **removed in React 19**. Use `createRoot(el).render(<App />)`, which also enables concurrent features and automatic batching.

i Note

StrictMode is a development-only helper. It intentionally **renders components twice** and **runs effects twice (mount, unmount, mount)** to expose impure code and missing cleanup. It does nothing in production builds. If you see a `console.log` twice or an API called twice in development, StrictMode is usually why.

1.8 React Versions at a Glance

✎ Definition

A timeline of major React releases and what each added, from JSX and the Virtual DOM to Hooks (16.8), concurrent rendering (18) and Actions and Server Components (19).

Version	Year	Highlights
React 0.x - 15	2013-2016	JSX, Virtual DOM, class components, createClass
16	2017	Fiber rewrite, fragments, portals, error boundaries, Context API (16.3), Hooks (16.8, 2019)
17	2020	New JSX transform (no need to import React), event delegation to root
18	2022	createRoot, concurrent rendering, automatic batching, useTransition, useDeferredValue, useId, Suspense on server, streaming SSR

Version	Year	Highlights
19	Dec 2024	Actions, useActionState, useOptimistic, useFormStatus, `use()`, ref as a prop, Server Components and Server Actions stable, document metadata, removed legacy APIs
19.1 / 19.2	2025	Owner stacks, `<Activity>`, `useEffectEvent`, React Compiler 1.0 (auto-memoization)

2. JavaScript Essentials for React

React is "just JavaScript". Most React bugs are really JavaScript misunderstandings. Master these first.

2.1 let, const and var

✎ Definition

Ways to declare variables: `let` (block-scoped, can be reassigned), `const` (block-scoped, cannot be reassigned) and the legacy `var` (function-scoped and hoisted).

```
const API_URL = '/api';     // cannot be reassigned (default choice)
let count = 0;              // can be reassigned
var old = 1;                // function-scoped, hoisted - avoid

const user = { name: 'Ali' };
user.name = 'Sam';          // ALLOWED: const protects the binding, not the object
// user = {};               // TypeError: Assignment to constant variable
```

⚡ Tricky Point

`const` does not make objects immutable. That's exactly why React needs you to *create new objects* for state updates rather than relying on `const`.

2.2 Arrow Functions

✎ Definition

A short function syntax (`(a, b) => a + b`) with implicit return for single expressions. Arrow functions do **not** have their own `this`; they use the surrounding one.

```
function add(a, b) { return a + b; }
const add2 = (a, b) => a + b;                 // implicit return
const square = x => x * x;                     // single param: no parens needed
const makeUser = (name) => ({ name, age: 0 }); // returning an object: wrap in ( )
const log = () => { console.log('hi'); };      // block body: needs explicit return
```

⚡ Tricky Point

Arrow functions don't have their own `this` (they use the surrounding one). This is why class components needed `this.handleClick = this.handleClick.bind(this)` for normal methods but not for arrow-function class fields.

2.3 Destructuring

✎ Definition

Destructuring unpacks values from arrays or properties from objects into variables in one statement: `const { name, age } = user`, `const [a, b] = arr`.

```
const user = { id: 1, name: 'Ali', address: { city: 'Pune' } };
const { name, address: { city } } = user;        // name='Ali', city='Pune'
const { role = 'guest' } = user;                 // default value
const { id: userId } = user;                     // rename

const [first, second, ...rest] = [10, 20, 30, 40];   // rest = [30, 40]
const [state, setState] = useState(0);               // array destructuring in hooks

function Card({ title, children, size = 'md' }) { }  // destructuring props
```

2.4 Spread and Rest

✎ Definition

The `...` operator. **Spread** expands an array or object into elements or properties (used for copying and merging). **Rest** collects the remaining elements or properties into one variable.

```
const a = [1, 2]; const b = [...a, 3];               // [1,2,3]  copy + add
const user = { name: 'Ali', age: 30 };
const updated = { ...user, age: 31 };                 // copy + override
const merged = { ...defaults, ...options };          // later keys win

function sum(...nums) { return nums.reduce((t, n) => t + n, 0); }   // rest params
const { password, ...safeUser } = user;              // omit a key
<Button {...props} />                                  // spread props in JSX
```

⚡ Tricky Point

Spread is a **shallow copy**. `{ ...user }` copies the top level; nested objects (`user.address`) are still shared. Updating nested state requires spreading every level you change.

2.5 Template Literals, Optional Chaining, Nullish Coalescing

✎ Definition

Template literals (`` `Hi ${name}` ``) build strings with embedded expressions. **Optional chaining** (`a?.b`) safely reads properties that may be missing. **Nullish coalescing** (`a ?? b`) gives a default only for null or undefined.

```
const msg = `Hello ${name}, you have ${items.length} items`;
const city = user?.address?.city;            // undefined instead of crash
const first = list?.[0];
onClose?.();                                  // call only if defined
const pageSize = settings.pageSize ?? 20;    // only null/undefined fallback
const label = input || 'Default';            // ANY falsy fallback ('' and 0 too!)
```

> **⚡ Tricky Point**
>
> `0 || 20` gives 20, but `0 ?? 20` gives 0. Use `??` when 0, '' or false are valid values (page numbers, amounts, toggles).

2.6 Truthy and Falsy

> **✎ Definition**
>
> In a boolean context, JavaScript treats some values as **falsy** (`false`, `0`, `''`, `null`, `undefined`, `NaN`) and everything else as **truthy**. This matters for conditional rendering.

Falsy values: `false, 0, -0, 0n, '', null, undefined, NaN`. **Everything else is truthy**, including `'0'`, `'false'`, `[]` and `{}`.

```
if ([]) console.log('runs!');        // empty array is truthy
Boolean('0');                         // true
```

2.7 Array Methods You Will Use Daily

> **✎ Definition**
>
> The array methods used constantly in React for rendering and immutable updates: `map`, `filter`, `find`, `reduce`, `some`, `includes`, `slice` and `toSorted`.

```
const txns = [
  { id: 1, amount: 500, status: 'SUCCESS' },
  { id: 2, amount: 1500, status: 'FAILED' },
  { id: 3, amount: 2500, status: 'SUCCESS' },
];
txns.map(t => t.amount);                      // [500, 1500, 2500]  - render lists
txns.filter(t => t.status === 'SUCCESS');     // remove items immutably
txns.find(t => t.id === 2);                   // first match or undefined
txns.findIndex(t => t.id === 2);              // 1
txns.some(t => t.amount > 2000);              // true
```

```
txns.every(t => t.amount > 0);                // true
txns.reduce((sum, t) => sum + t.amount, 0);   // 4500
txns.includes(x); txns.at(-1);                // last item
[...txns].sort((a, b) => b.amount - a.amount); // copy, then sort
txns.toSorted((a, b) => a.amount - b.amount); // ES2023 non-mutating sort
txns.toReversed(); txns.with(0, newItem);     // non-mutating helpers
Object.groupBy(txns, t => t.status);          // { SUCCESS: [...], FAILED: [...] }
```

Mutating (avoid on state)	Non-mutating (safe for state)
`push, pop, shift, unshift`	`[...arr, x]`, `arr.slice()`
`splice`	`filter`, `slice`, `toSpliced`
`sort, reverse`	`[...arr].sort()`, `toSorted`, `toReversed`
`arr[i] = x`	`arr.map((v, idx) => idx === i ? x : v)`, `arr.with(i, x)`

> **⚡ Tricky Point**
>
> `[10, 9, 1].sort()` returns `[1, 10, 9]` because the default sort compares **strings**. Always pass a comparator for numbers: `sort((a, b) => a - b)`.

2.8 Equality and References

✎ Definition

=== compares primitives by value but objects and arrays by **reference**, meaning whether they are the same object in memory. React uses this (Object.is) to decide whether state or props changed.

```
1 === 1;                        // true
'a' === 'a';                    // true
{} === {};                      // false - two different objects
[] === [];                      // false
const a = { x: 1 }; const b = a;
a === b;                        // true - same reference
NaN === NaN;                    // false;  Object.is(NaN, NaN) -> true
0 == '0';                       // true (loose equality coerces) - always use ===
```

★ Interview Tip

This is the foundation of React's change detection. React compares state, props (in `memo`) and hook dependencies using `Object.is` - a reference check for objects. A new object `{...}` is always "different", and mutating an existing object is "the same", so React may not re-render.

2.9 Modules: import and export

✎ Definition

ES modules split code across files: `export` makes values available to other files, and `import` brings them in. There are default exports and named exports.

```
// math.js
export const PI = 3.14;                 // named export
export function add(a, b) { return a + b; }
export default function Calculator() {} // default export (one per file)

// App.js
import Calculator, { PI, add as sum } from './math';
import * as math from './math';
const Chart = lazy(() => import('./Chart'));   // dynamic import for code splitting
```

2.10 Closures

✎ Definition

A **closure** is a function that remembers the variables of the scope where it was created, even after that scope has finished. Every event handler and hook callback is a closure.

A closure is a function that remembers variables from the scope where it was created. Every event handler and effect in React is a closure over the props and state **of the render that created it**.

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const next = makeCounter();
next(); next();                 // 1, 2 - 'count' survives between calls
```

> **⚡ Tricky Point**
>
> **Stale closures** (Chapter 8) happen because a handler or effect created during render #1 still sees render #1's state values, even after state has changed.

2.11 Promises and async/await

> **✎ Definition**
>
> A **Promise** represents a value that will be available later (pending, then fulfilled or rejected). `async/await` is syntax for writing promise-based code that reads like synchronous code.

```
fetch('/api/txns')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

async function loadTxns() {
  try {
    const res = await fetch('/api/txns');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch does NOT reject on 404/500!
    return await res.json();
  } catch (err) {
    console.error(err);
  }
}
const [users, txns] = await Promise.all([getUsers(), getTxns()]);   // parallel
const results = await Promise.allSettled([a(), b()]);               // never rejects
```

> **⚡ Tricky Point**
>
> `fetch` only rejects on **network errors**. A 404 or 500 response resolves normally - always check `res.ok`. (axios, by contrast, throws for non-2xx statuses.)

2.12 The Event Loop in One Minute

> **✎ Definition**
>
> The **event loop** is how single-threaded JavaScript handles async work. It runs the call stack, then all **microtasks** (promises), then the next **macrotask** (timers, events).

JavaScript is single-threaded. Synchronous code runs first, then **microtasks** (promise callbacks), then **macrotasks** (setTimeout, events).

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
// Output: A D C B
```

3. JSX in Depth

3.1 What is JSX?

> ✎ **Definition**
>
> **JSX** is a syntax extension that lets you write HTML-like markup inside JavaScript. It compiles to `React.createElement` (or `jsx()`) calls that describe the UI.

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript. It is **not HTML** and **not a string**: it compiles to function calls that create React elements.

```
const el = <h1 className="title">Hello {name}</h1>;

// compiles to (new JSX transform, React 17+):
import { jsx as _jsx } from 'react/jsx-runtime';
const el2 = _jsx('h1', { className: 'title', children: ['Hello ', name] });

// classic transform:
const el3 = React.createElement('h1', { className: 'title' }, 'Hello ', name);
```

3.2 JSX Rules

> ✎ **Definition**
>
> The rules JSX enforces: return a single root element, close every tag, use `className` and camelCase attributes, and put JavaScript expressions inside `{ }`.

- **Return a single root element.** Wrap siblings in a `<div>` or a **Fragment** `<>...</>`.
- **Close every tag:** `<img />`, `<br />`, `<input />`.
- **camelCase attributes:** `className` (not class), `htmlFor` (not for), `onClick`, `tabIndex`, `strokeWidth`. Exceptions: `aria-*` and `data-*` stay hyphenated.
- **JavaScript goes in curly braces** `{ }`: any *expression* (value-producing) is allowed, but not statements like `if`, `for` or `const`.
- **Component names start with a capital letter.** `<button>` is an HTML tag; `<Button>` is your component.
- **Comments:** `{/* comment */}`.

```
function Profile({ user }) {
  return (
    <>
      <label htmlFor="email">Email</label>
      <input id="email" type="email" defaultValue={user.email} />
      {/* expressions only */}
      <p>{user.firstName + ' ' + user.lastName}</p>
      <p>{user.isAdmin ? 'Admin' : 'User'}</p>
      <img src={user.avatar} alt="" style={{ width: 40, borderRadius: '50%' }} />
    </>
  );
}
```

> **⚡ Tricky Point**
>
> `style` takes an **object** with camelCase keys: `style={{ backgroundColor: 'red', fontSize: 14 }}`. The outer braces mean "JavaScript", the inner braces are the object. Numbers get `px` automatically for most properties.

> **⚡ Tricky Point**
>
> Lowercase component names are treated as HTML tags: `<myButton />` renders an unknown `<mybutton>` element with no error. Always capitalise components.

3.3 Fragments

> **✎ Definition**
>
> A **Fragment** (<>...</> or <Fragment>) groups several elements without adding an extra DOM node.

```
<>
  <td>Name</td>
  <td>Amount</td>
</>
// Keyed fragment (short syntax can't take keys):
{items.map(item => (
  <Fragment key={item.id}>
    <dt>{item.term}</dt>
    <dd>{item.desc}</dd>
  </Fragment>
))}
```

3.4 Conditional Rendering

> **✎ Definition**
>
> Showing different UI depending on a condition, using `if`, the ternary `? :`, or `&&` inside JSX (watch out for `0` being rendered).

```
function Status({ isLoading, error, txns }) {
  if (isLoading) return <Spinner />;              // early return
  if (error) return <ErrorBox message={error.message} />;

  return (
    <div>
      {txns.length > 0 ? <TxnTable rows={txns} /> : <Empty />}   {/* ternary */}
      {txns.length > 100 && <p>Showing first 100</p>}            {/* && short-circuit */}
      {user?.isAdmin && <AdminPanel />}
    </div>
  );
}
function Nothing() { return null; }          // render nothing
```

> **⚡ Tricky Point**
>
> **The `0` bug:** `{items.length && <List />}` renders a literal **0** on screen when the list is empty, because `0 && x` evaluates to `0` and React renders numbers. Use `{items.length > 0 && <List />}` or `!!items.length`. Values `false`, `null`, `undefined` and `true` render nothing; `0` and `NaN` do render.

3.5 Rendering Lists and Keys

✎ Definition

Rendering arrays with `map()`. Each item needs a stable, unique **key** prop so React can track which items were added, removed or reordered.

```
const txns = [{ id: 'T1', amount: 500 }, { id: 'T2', amount: 900 }];

<ul>
  {txns.map(t => (
    <li key={t.id}>{t.id}: {t.amount}</li>     // key on the OUTERMOST element of the map
  ))}
</ul>
```

Keys tell React which list item is which between renders, so it can match, reorder, insert and delete efficiently while preserving each item's state.

Rules for keys:

- Must be **unique among siblings** (not globally) and **stable** across renders.
- Use IDs from your data (database id, RRN, UUID).
- Don't use `Math.random()` or `crypto.randomUUID()` during render: new key every render = components re-created and state lost.
- Array **index** is acceptable only when the list is static (never reordered, filtered or inserted into).
- `key` is not passed as a prop: `props.key` is undefined. Pass it again under another name if the child needs it.

⚡ Tricky Point

Index-as-key bug: a list of inputs keyed by index. Delete the first item and the text you typed "moves" to the wrong row, because React matched component state by position. With stable IDs the state follows the right item.

★ Interview Tip

"Why are keys important?" - Keys give list items a stable identity so React's diff can match old and new items by key instead of position. This avoids unnecessary re-creation, preserves component state correctly and makes reordering efficient.

Resetting state with a key

Changing a component's `key` makes React throw away the old instance and mount a fresh one - a handy trick to reset a form.

```
<EditForm key={selectedUserId} user={selectedUser} />   // new user -> fresh form state
```

3.6 Rendering HTML Strings and Escaping

✎ Definition

React **escapes** all values rendered in JSX to prevent XSS. To insert raw HTML you must use `dangerouslySetInnerHTML`, and the HTML should be sanitised first.

```
const input = '<img src=x onerror=alert(1)>';
<p>{input}</p>                          // SAFE: React escapes it, shows text
<div dangerouslySetInnerHTML={{ __html: sanitizedHtml }} />   // raw HTML: sanitize first (DOMPurify)
```

⚠ Watch Out

React protects you from XSS by escaping values in `{ }`. `dangerouslySetInnerHTML`, `href={userInput}` (e.g. `javascript:` URLs) and direct DOM writes bypass this protection.

4. Components and Props

4.1 Function Components

> ✎ **Definition**
>
> A **function component** is a JavaScript function, named in PascalCase, that receives props and returns JSX describing part of the UI. It is the modern way to write React components.

A component is a **JavaScript function that returns JSX**. React calls it whenever it needs to know what to show.

```
function Welcome() {
  return <h2>Welcome to TraceNext</h2>;
}
const Footer = () => <footer>(c) 2026</footer>;

export default function App() {
  return (
    <>
      <Welcome />
      <Welcome />       {/* reuse as many times as you like */}
      <Footer />
    </>
  );
}
```

> ⚠ **Watch Out**
>
> Never define a component **inside** another component. Each render creates a brand-new component type, so React unmounts and remounts it every time: state is lost, inputs lose focus and performance suffers. Define components at the top level of the module.

4.2 Props

> ✎ **Definition**
>
> **Props** (properties) are the **read-only** inputs a parent passes to a child component, like function arguments. They configure the child's output.

Props (properties) are the **inputs** of a component, passed like HTML attributes. They are **read-only**.

```
function TxnRow({ id, amount, status = 'PENDING', onSelect }) {    // destructure + default
  return (
    <tr onClick={() => onSelect(id)}>
      <td>{id}</td>
      <td>{amount.toLocaleString('en-IN', { style: 'currency', currency: 'INR' })}</td>
      <td className={status === 'FAILED' ? 'text-red' : ''}>{status}</td>
    </tr>
  );
}
<TxnRow id="T1" amount={2500} status="SUCCESS" onSelect={handleSelect} />
<TxnRow id="T2" amount={100} />                      // status defaults to PENDING
<Toggle enabled />                                    // shorthand for enabled={true}
<TxnRow {...txn} onSelect={handleSelect} />           // spread an object as props
```

> **⚡ Tricky Point**
>
> **Props are immutable** from the child's point of view: `props.amount = 5` is a bug (and throws in strict mode for frozen props). To change data, the *owner* of the data (usually the parent) updates its state and passes new props down.

> **⚡ Tricky Point**
>
> Strings can be passed with quotes (`title="Hi"`), but **everything else needs braces**: `count={5}`, `active={false}`, `items={[1, 2]}`. Writing `count="5"` passes the *string* "5".

4.3 The children Prop and Composition

> **✎ Definition**
>
> `children` is a special prop containing whatever is placed between a component's tags. **Composition** builds complex UIs by nesting components rather than through inheritance.

Whatever you put between a component's opening and closing tags arrives as `props.children`.

```
function Card({ title, children, footer }) {
  return (
    <section className="card">
      <h3>{title}</h3>
      <div className="card-body">{children}</div>
      {footer && <div className="card-footer">{footer}</div>}
    </section>
  );
}
<Card title="Settlement Summary" footer={<button>Export</button>}>
  <p>Total: 4,500</p>
  <TxnTable rows={rows} />
</Card>
```

> **★ Interview Tip**
>
> React favours **composition over inheritance**. Instead of extending components, you combine them by passing elements as `children` or as named props ("slots"). Meta reports that they have never needed component inheritance hierarchies.

4.4 One-Way Data Flow

> ✎ **Definition**
>
> In React, data flows **downward** from parent to child through props. Children communicate upward by calling callback functions that the parent passes as props.

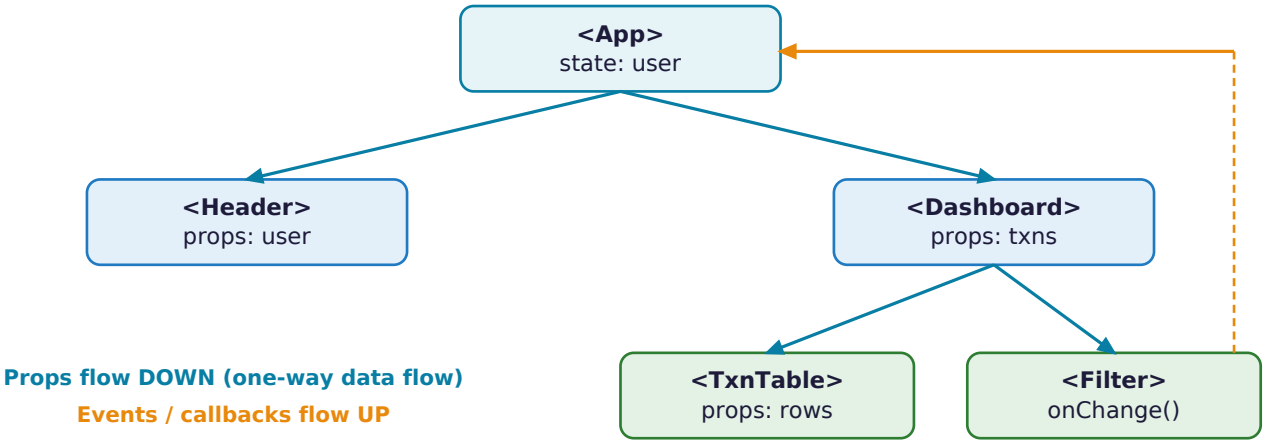


Figure 4.1 - Data flows down via props; events flow up via callbacks

```
function Dashboard() {
  const [filter, setFilter] = useState('ALL');               // state lives in the parent
  const visible = txns.filter(t => filter === 'ALL' || t.status === filter);
  return (
    <>
      <FilterBar value={filter} onChange={setFilter} />      // pass callback down
      <TxnTable rows={visible} />
    </>
  );
}
function FilterBar({ value, onChange }) {
  return (
    <select value={value} onChange={e => onChange(e.target.value)}>   // child calls up
      <option>ALL</option><option>SUCCESS</option><option>FAILED</option>
    </select>
  );
}
```

4.5 Pure Components

> ✎ **Definition**
>
> A **pure component** always returns the same JSX for the same props and state, and has no side effects during rendering. React relies on this for correctness and optimisation.

React assumes components behave like **pure functions**: same props, state and context in, same JSX out, and **no side effects during rendering**.

```
let guestCount = 0;
function Bad() {
  guestCount++;                          // mutates outside variable during render - impure!
  return <p>Guest #{guestCount}</p>;  // shows different numbers in StrictMode
}
function Good({ guestNo }) {
  return <p>Guest #{guestNo}</p>;     // pure
}
```

Side effects (API calls, timers, DOM manipulation, logging, subscriptions) belong in **event handlers** or **effects**, never in the render body. Creating and mutating *local* variables inside render is fine.

4.6 Prop Validation: TypeScript and PropTypes

✎ Definition

Checking that components receive props of the right type: **TypeScript** types at compile time (preferred), or the legacy run-time **PropTypes** library.

```
// TypeScript (recommended)
type TxnRowProps = {
  id: string;
  amount: number;
  status?: 'SUCCESS' | 'FAILED' | 'PENDING';
  onSelect: (id: string) => void;
  children?: React.ReactNode;
};
function TxnRow({ id, amount, status = 'PENDING', onSelect }: TxnRowProps) { /* ... */ }
```

i Note

`PropTypes` and `defaultProps` for function components were **removed in React 19** (they are silently ignored). Use TypeScript for types and JavaScript default parameters for defaults.

4.7 Component Design Tips

✎ Definition

Guidelines for good components: keep them small and single-purpose, name them clearly, lift state only as high as needed, and prefer composition and clear prop interfaces.

- One component = one responsibility. If a file grows past ~200 lines, split it.
- **Presentational** components (how things look, receive props) vs **container/feature** components (fetch data, hold state). Custom hooks now often replace containers.
- Name event props `onSomething` and the handlers `handleSomething`.
- Keep state as **local** as possible and as **high** as necessary.
- Prefer **controlled** components for inputs that other components care about.

5. State and useState

5.1 What is State?

> ✎ **Definition**
>
> **State** is data a component remembers between renders that changes over time. Updating it with the setter from `useState` causes the component to re-render.

State is data that **belongs to a component, changes over time and affects what is rendered**. When state changes, React re-renders the component.

> ☼ **Simple Analogy**
>
> Props are like the arguments given to a function by its caller. State is like the component's own memory or notebook: it persists between renders, and only the component itself can write in it.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);     // [current value, setter function]
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Props	State
Passed in from the parent	Owned and managed inside the component
Read-only for the component	Changed with the setter function
Changing them = parent re-renders with new values	Changing it = this component re-renders
Like function parameters	Like a component's private memory

Why not a normal variable?

```
function Broken() {
  let count = 0;                                    // reset to 0 on every render
  return <button onClick={() => { count++; }}>{count}</button>;   // never updates the UI
}
```

A local variable is (1) re-created on every render and (2) changing it doesn't tell React to re-render. `useState` solves both.

5.2 State is a Snapshot

✎ Definition

During a render, a state variable is a fixed **snapshot**. Calling the setter does not change the current value; it schedules a re-render that uses the new value.

Calling `setCount` doesn't change `count` in the current render. It **requests a new render** in which `count` will have the new value.

```
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);       // schedule: 0 + 1
    setCount(count + 1);       // schedule: 0 + 1  (count is still 0 in THIS render)
    setCount(count + 1);       // schedule: 0 + 1
    console.log(count);        // 0 - not updated yet!
  }
  // After click: count is 1, not 3
}
```

Functional updates (updater function)

```
function handleClick() {
  setCount(c => c + 1);        // 0 -> 1
  setCount(c => c + 1);        // 1 -> 2
  setCount(c => c + 1);        // 2 -> 3   => count is 3
}
```

⚡ Tricky Point

Use the **updater form** `setX(prev => ...)` whenever the new state depends on the previous state, especially inside timers, intervals, async callbacks and effects - it always receives the latest value and avoids stale closures.

★ Interview Tip

"Is setState synchronous or asynchronous?" - It's neither in the usual sense: the setter *schedules* an update. React batches updates and re-renders afterwards, and the current render's variable never changes (state is a snapshot). Read the new value in the next render, or compute it yourself first.

5.3 Batching

✎ Definition

Batching means React groups several state updates made in the same event (in React 18+, everywhere) into a single re-render, for better performance.

React groups multiple state updates into **a single re-render**. Since React 18, batching is **automatic everywhere**: event handlers, promises, `setTimeout`, native event listeners.

```
function handleSave() {
  setLoading(true);
  setError(null);
  setCount(c => c + 1);      // ONE re-render, not three
}
setTimeout(() => { setA(1); setB(2); }, 1000);   // React 18+: also one render
import { flushSync } from 'react-dom';
flushSync(() => setOpen(true));                   // rare: force synchronous update (e.g. before measuring DOM)
```

5.4 Lazy Initial State

✎ Definition

Passing a **function** to `useState(() => expensive())` so the expensive initial value is computed only on the first render, not on every render.

```
const [rows, setRows] = useState(expensiveParse(csv));     // runs EVERY render (result ignored after first)
const [rows2, setRows2] = useState(() => expensiveParse(csv)); // runs only on the first render
const [theme, setTheme] = useState(() => localStorage.getItem('theme') ?? 'light');
```

⚡ Tricky Point

Initial state is used **only on the first render**. `useState(props.value)` does **not** update when `props.value` changes later. If you need that, either use the prop directly (don't copy it into state), or reset with a `key`.

5.5 Updating Objects Immutably

✎ Definition

Never mutate state objects. Create a **new object** with the changes (`{ ...obj, key: value }`) so React sees a new reference and re-renders.

State must be treated as **read-only**. Create a new object instead of mutating.

```
const [form, setForm] = useState({ name: '', email: '', address: { city: '', pin: '' } });

form.name = 'Ali';                // WRONG: mutation, same reference -> React may not re-render
setForm(form);                    // Object.is(old, new) is true -> render skipped

setForm({ ...form, name: 'Ali' });                          // correct
setForm(f => ({ ...f, [e.target.name]: e.target.value }));   // generic input handler
setForm(f => ({ ...f, address: { ...f.address, city: 'Pune' } }));  // nested: spread each level
```

5.6 Updating Arrays Immutably

✎ Definition

Updating state arrays by creating **new arrays**: `[...arr, x]` to add, `filter` to remove, `map` to update, `toSorted` to sort. Avoid `push`, `splice` and in-place `sort`.

```
const [items, setItems] = useState([]);

setItems([...items, newItem]);                      // add to end
setItems([newItem, ...items]);                      // add to start
setItems(items.filter(i => i.id !== id));           // remove
setItems(items.map(i => i.id === id ? { ...i, done: !i.done } : i));   // update one
setItems([...items.slice(0, idx), newItem, ...items.slice(idx)]);      // insert at index
setItems([...items].sort((a, b) => a.amount - b.amount));              // sort a copy
setItems([...items].reverse());
```

⚡ Tricky Point

`items.map(i => { i.done = true; return i; })` creates a new **array** but mutates the **objects** inside it - shared with the previous state. Memoized children receiving those objects won't update, and "previous state" is corrupted. Always copy the item: `{ ...i, done: true }`.

Immer for deeply nested updates

```
import { useImmer } from 'use-immer';
const [order, updateOrder] = useImmer(initialOrder);
updateOrder(draft => { draft.items[2].qty += 1; });   // write "mutating" code; Immer produces a new object
```

Redux Toolkit uses Immer internally, which is why you can "mutate" inside its reducers.

5.7 Lifting State Up

✎ Definition

Lifting state up means moving shared state to the closest common parent of the components that need it, then passing it down as props along with updater callbacks.

When two components need the same data, move the state to their **closest common parent** and pass it down.

```
function Accordion() {
  const [activeIndex, setActiveIndex] = useState(0);
  return (
    <>
      <Panel title="About" isActive={activeIndex === 0} onShow={() => setActiveIndex(0)}>...</Panel>
      <Panel title="Fees" isActive={activeIndex === 1} onShow={() => setActiveIndex(1)}>...</Panel>
    </>
  );
}
function Panel({ title, isActive, onShow, children }) {
  return (
    <section>
      <h3>{title}</h3>
      {isActive ? <p>{children}</p> : <button onClick={onShow}>Show</button>}
    </section>
  );
}
```

5.8 Derived State: Don't Store What You Can Calculate

✎ Definition

Derived state is a value that can be calculated from existing props or state. Compute it during render instead of storing it in separate state, which can get out of sync.

```
// BAD: duplicate state that must be kept in sync manually
const [txns, setTxns] = useState([]);
const [total, setTotal] = useState(0);
const [filtered, setFiltered] = useState([]);

// GOOD: store the minimum; compute the rest during render
const [txns, setTxns] = useState([]);
const [query, setQuery] = useState('');
const filtered = txns.filter(t => t.id.includes(query));
const total = filtered.reduce((s, t) => s + t.amount, 0);
const [selectedId, setSelectedId] = useState(null);       // store the ID...
const selected = txns.find(t => t.id === selectedId);     // ...not a copy of the object
```

★ Interview Tip

Principles for structuring state: group related state, avoid contradictions (use one `status` string instead of `isLoading` + `isError` + `isSuccess` booleans), avoid redundant/derived state, avoid duplication, and avoid deep nesting (normalise by ID).

5.9 State Preservation and Position in the Tree

✎ Definition

React keeps a component's state as long as the **same component type** is rendered at the **same position** in the tree. Changing the type or the key resets it.

React keeps state tied to a component's **position in the UI tree**, not to the variable or JSX line.

```
{isPremium ? <Counter /> : <Counter />}        // SAME position + type -> state is KEPT when toggling
{isPremium ? <Counter key="p" /> : <Counter key="n" />}  // different keys -> state RESET
{isPremium ? <Counter /> : <p>Basic</p>}       // different type -> Counter unmounted, state lost
```

⚡ Tricky Point

State is preserved as long as the same component type renders at the same position. Removing it, changing its type, or changing its `key` destroys its state.

6. Events and Forms

6.1 Handling Events

✎ Definition

Responding to user actions by passing a **function** to camelCase event props such as `onClick` and `onChange`: `onClick={handleClick}`, not `handleClick()`.

```
function Toolbar() {
  function handleExport(e) {
    console.log('Exporting', e.type);        // e is a SyntheticEvent
  }
  return (
    <>
      <button onClick={handleExport}>Export</button>            {/* pass the function */}
      <button onClick={() => remove(42)}>Delete</button>         {/* need args: wrap in arrow */}
      <input onChange={e => setQuery(e.target.value)} onKeyDown={e => e.key === 'Enter' && search()} />
    </>
  );
}
```

⚡ Tricky Point

`onClick={handleClick()}` **calls** the function during render (and assigns its return value as the handler) - often causing an infinite render loop if it sets state. Pass the function itself: `onClick={handleClick}` or `onClick={() => handleClick(id)}`.

6.2 Synthetic Events

✎ Definition

A **SyntheticEvent** is React's cross-browser wrapper around the native DOM event, with the same interface (preventDefault, stopPropagation, target).

React wraps native browser events in **SyntheticEvent** objects with a consistent cross-browser API (`e.target`, `e.preventDefault()`, `e.stopPropagation()`, `e.nativeEvent`). Since React 17, React attaches listeners at the **root container** (not `document`) and dispatches them through the component tree.

Common event props	Notes
`onClick, onDoubleClick, onContextMenu`	Mouse
`onChange`	Fires on **every keystroke** for inputs (unlike native change)
`onInput, onKeyDown, onKeyUp`	Keyboard (`onKeyPress` is deprecated)
`onFocus, onBlur`	These **do** bubble in React
`onSubmit, onReset`	Forms
`onMouseEnter, onMouseLeave`	Don't bubble
`onScroll`	Does not bubble in React 17+
`onClickCapture`	Capture phase variant of any event

Propagation and default behaviour

```
<div onClick={() => console.log('row clicked')}>
  <button onClick={e => { e.stopPropagation(); edit(); }}>Edit</button>   // row handler won't run
</div>
<form onSubmit={e => { e.preventDefault(); save(); }}>   // stop the page reload
```

> ⚡ **Tricky Point**
>
> Default `type` of a `<button>` inside a `<form>` is **submit**. A "Cancel" button without `type="button"` will submit the form. Always set `type="button"` on non-submit buttons.

6.3 Controlled Components

> ✎ **Definition**
>
> A **controlled component** is a form input whose value is driven by React state (`value` + onChange). React state is the single source of truth.

In a **controlled** input, React state is the single source of truth: `value` comes from state and every change goes through `onChange`.

```
function LoginForm({ onLogin }) {
  const [form, setForm] = useState({ email: '', password: '', remember: false, role: 'user' });
  const [errors, setErrors] = useState({});

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm(f => ({ ...f, [name]: type === 'checkbox' ? checked : value }));   // one handler for all
  }

  function validate() {
    const errs = {};
    if (!form.email.includes('@')) errs.email = 'Invalid email';
    if (form.password.length < 8) errs.password = 'Min 8 characters';
    return errs;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const errs = validate();
    setErrors(errs);
    if (Object.keys(errs).length === 0) onLogin(form);
  }

  return (
    <form onSubmit={handleSubmit} noValidate>
      <input name="email" value={form.email} onChange={handleChange} />
      {errors.email && <span className="err">{errors.email}</span>}
      <input name="password" type="password" value={form.password} onChange={handleChange} />
      <label><input name="remember" type="checkbox" checked={form.remember} onChange={handleChange} /> Remember</label>
      <select name="role" value={form.role} onChange={handleChange}>
        <option value="user">User</option><option value="admin">Admin</option>
      </select>
      <textarea name="note" value={form.note ?? ''} onChange={handleChange} />
      <button type="submit">Login</button>
    </form>
  );
}
```

> ⚡ **Tricky Point**
>
> **Controlled vs uncontrolled warning:** `value={undefined}` makes an input uncontrolled; later giving it a string makes it controlled, and React warns. Always initialise with `''` (not `undefined`/`null`). A `value` without `onChange` makes the input read-only (use `defaultValue` or `readOnly`).

6.4 Uncontrolled Components

> ✎ **Definition**
>
> An **uncontrolled component** keeps its own value in the DOM. React reads it when needed through a `ref` or FormData, and sets only the `defaultValue`.

The DOM keeps the value; you read it when needed with a **ref** or from **FormData**.

```
function QuickSearch({ onSearch }) {
  const inputRef = useRef(null);
  return (
    <form onSubmit={e => { e.preventDefault(); onSearch(inputRef.current.value); }}>
      <input ref={inputRef} defaultValue="" />
      <input type="file" name="upload" />   {/* file inputs are ALWAYS uncontrolled */}
    </form>
  );
}
function handleSubmit(e) {
  e.preventDefault();
  const data = Object.fromEntries(new FormData(e.currentTarget));   // { email: '...', ... }
}
```

Controlled	Uncontrolled
Value in React state	Value in the DOM
`value` + `onChange`	`defaultValue` + `ref` / FormData
Instant validation, formatting, conditional UI	Less code, fewer re-renders
Re-render on every keystroke	Read on submit

6.5 React 19 Form Actions

> ✎ **Definition**
>
> In React 19, a **form action** is a function passed to `<form action={fn}>`. React handles submission, pending state (`useFormStatus`) and results (`useActionState`) for you.

React 19 lets you pass a **function** to `<form action>`. React calls it with the `FormData`, handles pending state, and resets uncontrolled fields after success.

```
import { useActionState } from 'react';
import { useFormStatus } from 'react-dom';

async function createTicket(prevState, formData) {
  const title = formData.get('title');
  if (!title) return { error: 'Title required' };
  await api.post('/tickets', { title });
  return { error: null, ok: true };
}
function SubmitButton() {
  const { pending } = useFormStatus();          // reads the parent form's status
  return <button disabled={pending}>{pending ? 'Saving...' : 'Save'}</button>;
}

export function TicketForm() {
  const [state, formAction, isPending] = useActionState(createTicket, { error: null });
  return (
    <form action={formAction}>
      <input name="title" />
      {state.error && <p className="err">{state.error}</p>}
      <SubmitButton />
    </form>
  );
}
```

6.6 Form Libraries

> **✎ Definition**
>
> Libraries that simplify complex forms: **React Hook Form** (fast, uncontrolled-input based) together with a schema validator such as **Zod** for validation rules.

For large forms, **React Hook Form** (uncontrolled, minimal re-renders) with **Zod** schema validation is the most common choice.

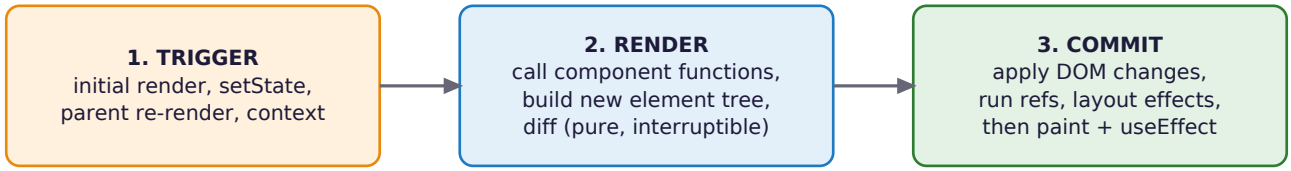
```
const schema = z.object({ email: z.string().email(), amount: z.coerce.number().positive() });
function PaymentForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm({ resolver: zodResolver(schema) });
  return (
    <form onSubmit={handleSubmit(data => api.pay(data))}>
      <input {...register('email')} />
      {errors.email && <span>{errors.email.message}</span>}
      <input {...register('amount')} />
      <button disabled={isSubmitting}>Pay</button>
    </form>
  );
}
```

7. Rendering, Re-rendering and Reconciliation

7.1 Trigger, Render, Commit

✎ Definition

The three steps of every UI update: a **trigger** (initial render or state change), the **render** phase (React calls components to compute the new UI) and the **commit** phase (React applies changes to the DOM).



Render = calculating what the UI should look like. Commit = touching the real DOM.

A render does NOT always mean the DOM changed. If nothing differs, commit does nothing.

Figure 7.1 - The three phases of every UI update

☼ Simple Analogy

React is like a restaurant kitchen. **Trigger:** a customer places (or changes) an order. **Render:** the chef prepares the dish (calls your components). **Commit:** the waiter places the dish on the table (updates the DOM). If the dish is identical to what is already on the table, nothing is swapped.

7.2 When Does a Component Re-render?

✎ Definition

A component re-renders when its **state** changes, its **parent** re-renders, or a **context** it uses changes. Prop changes alone are not the trigger; the parent's re-render is.

A component re-renders when:

- Its **state** changes (via a setter with a new value).
- Its **parent re-renders** (by default, all children re-render, whether props changed or not).
- A **context** it consumes changes.
- A hook it uses (custom hook, external store) triggers an update.

```
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      <ExpensiveChild />      {/* re-renders on every click, even with no props! */}
    </>
  );
}
```

⚡ Tricky Point

Props changing does NOT cause a re-render by itself - the parent re-rendering does. And setting state to the **same value** (`Object.is` equal) lets React bail out without re-rendering children.

★ Interview Tip

"Does a re-render mean the DOM is updated?" - No. Re-rendering means React called the component and produced new elements. React then diffs them; if nothing changed, the DOM is untouched. Re-renders are usually cheap; optimise only when measured.

7.3 Reconciliation and the Diffing Algorithm

✎ Definition

Reconciliation is how React compares the new element tree with the previous one. Its O(n) **diffing** algorithm assumes different element types produce different trees and uses keys to match list items.

Comparing two arbitrary trees is O(n^3). React uses **heuristics** to make it **O(n)**:

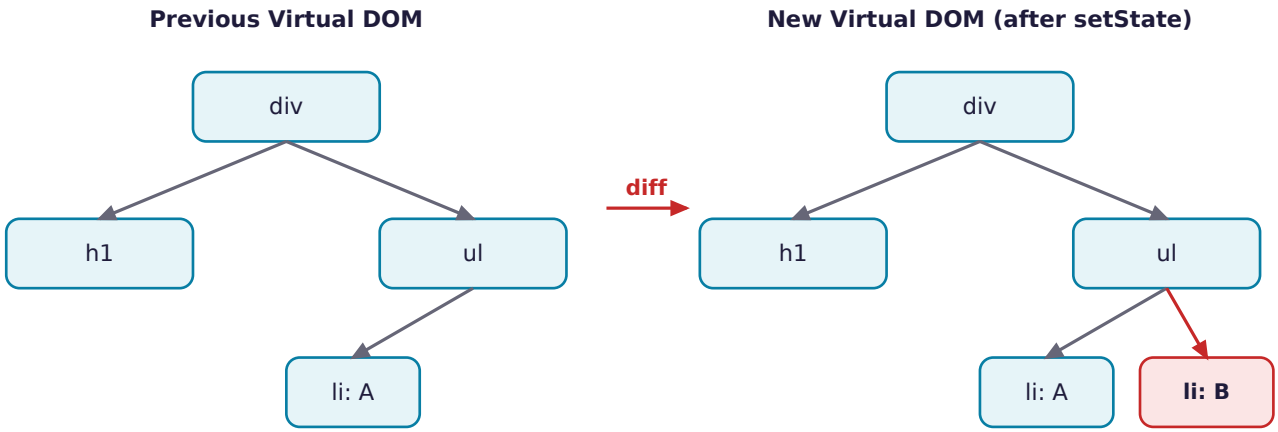


Figure 7.2 - React compares the new tree with the previous one

1. **Different element types produce different trees.** If `<div>` becomes `<section>`, or `<Counter>` becomes `<Timer>`, React destroys the old subtree (unmounting components, losing state) and builds a new one. 2. **Same type DOM element:** React keeps the DOM node and updates only changed attributes. 3. **Same type component:** the instance (and its state) is kept; React re-renders it with new props. 4. **Lists use keys:** children are matched by `key` instead of position.

Composition trick: avoid re-rendering expensive children

```
function ColorPicker({ children }) {                // owns the frequently-changing state
  const [color, setColor] = useState('red');
  return <div style={{ color }}><input onChange={e => setColor(e.target.value)} />{children}</div>;
}
<ColorPicker>
  <ExpensiveTree />     {/* created by the PARENT, so it doesn't re-render when color changes */}
</ColorPicker>
```

7.4 Mounting, Updating, Unmounting

> ✎ **Definition**
>
> The lifecycle of a component: **mounting** (first added to the screen), **updating** (re-rendering with new props or state) and **unmounting** (removed from the screen).

- **Mount:** component appears in the tree for the first time (state initialised, effects run).
- **Update:** re-render with new state/props (effects with changed dependencies re-run).
- **Unmount:** removed from the tree (effect cleanups run, state destroyed).

7.5 Class Components and Lifecycle Methods (Legacy)

> ✎ **Definition**
>
> The older way to write components, as ES6 classes with `this.state` and **lifecycle methods** (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`). Still found in legacy code and error boundaries.

You will still see class components in older codebases and interviews. New code should use function components and hooks.

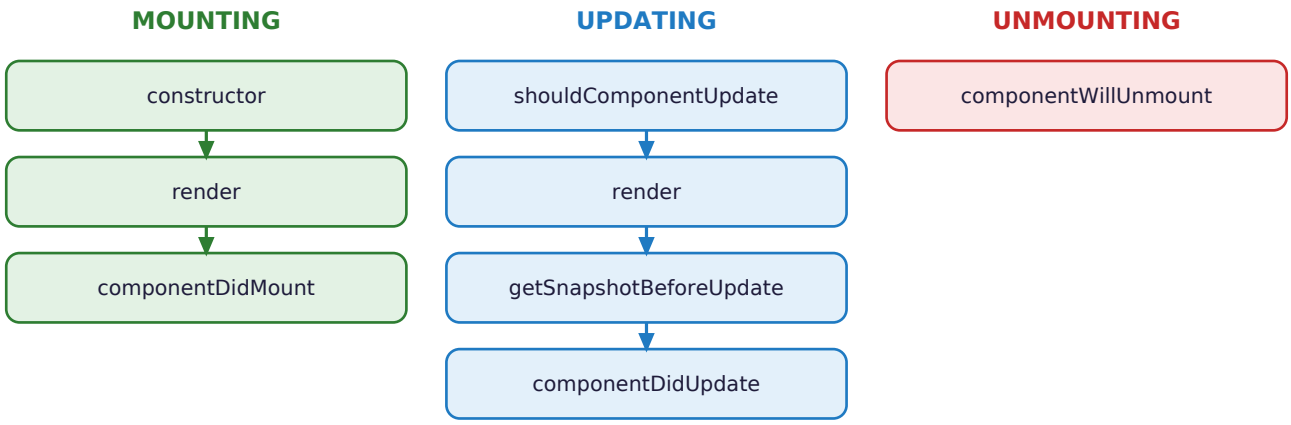


Figure 7.3 - Class component lifecycle

```
class TxnList extends React.Component {
  state = { txns: [], loading: true };           // class field

  componentDidMount() {                          // after first render: fetch, subscribe
    this.load();
    this.timer = setInterval(this.load, 30000);
  }
  componentDidUpdate(prevProps, prevState) {     // after updates
    if (prevProps.bankId !== this.props.bankId) this.load();
  }
  componentWillUnmount() {                       // cleanup
    clearInterval(this.timer);
  }
  shouldComponentUpdate(nextProps, nextState) {  // return false to skip render
    return nextState.txns !== this.state.txns;
  }
  load = async () => {                           // arrow field: 'this' is bound
    const txns = await api.getTxns(this.props.bankId);
    this.setState({ txns, loading: false });     // merges into state (hooks REPLACE)
  };
  render() {
    const { txns, loading } = this.state;
    return loading ? <Spinner /> : <Table rows={txns} />;
  }
}
```

Class lifecycle	Hooks equivalent
`constructor` / state init	`useState(initial)` / lazy initialiser
`componentDidMount`	`useEffect(fn, [])`
`componentDidUpdate`	`useEffect(fn, [deps])`
`componentWillUnmount`	cleanup function returned from `useEffect`
`shouldComponentUpdate` / `PureComponent`	`React.memo`
`getDerivedStateFromProps`	compute during render / `key` reset
`getSnapshotBeforeUpdate`	`useLayoutEffect` (approximately)
`componentDidCatch` / `getDerivedStateFromError`	No hook yet - still need a class Error Boundary
`this.setState(obj)` merges	`setState` replaces (spread manually)

⚡ Tricky Point

`componentWillMount`, `componentWillReceiveProps` and `componentWillUpdate` are deprecated (prefixed `UNSAFE_`) because they're unsafe with concurrent rendering.

★ Interview Tip

"Hooks vs class components?" - Hooks let function components use state and lifecycle features without classes: less boilerplate, no `this` binding, logic grouped by concern instead of split across lifecycle methods, and reusable stateful logic via custom hooks instead of HOCs/render props.

8. useEffect and Side Effects

8.1 What is an Effect?

> ✎ **Definition**
>
> An **effect** (`useEffect`) runs code **after render** to synchronise a component with an external system (network, subscriptions, timers, DOM APIs), with optional cleanup.

Effects let a component **synchronise with something outside React**: network requests, subscriptions, timers, browser APIs, third-party widgets, manual DOM changes. They run **after** React has committed changes to the screen.



useLayoutEffect runs after commit but BEFORE paint (blocks painting). On unmount, cleanup runs one last time.

Figure 8.1 - When effects run

```
useEffect(() => {
  // setup: runs after render is committed
  return () => {
    // cleanup: runs before the next effect run and on unmount
  };
}, [dependencies]);
```

Dependency array	When the effect runs
omitted `useEffect(fn)`	After **every** render
`[]` empty	Once after mount (cleanup on unmount)
`[a, b]`	After mount and whenever `a` or `b` changed (`Object.is`)

8.2 Common Examples

✎ Definition

Typical uses of effects: subscribing to events, setting up timers, connecting to a chat or WebSocket, and integrating non-React widgets, each with matching cleanup.

```
// 1) Document title
useEffect(() => { document.title = `${unread} new alerts`; }, [unread]);

// 2) Subscriptions with cleanup
useEffect(() => {
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener('resize', onResize);
  return () => window.removeEventListener('resize', onResize);   // must remove the SAME function
}, []);
// 3) Timers
useEffect(() => {
  const id = setInterval(() => setSeconds(s => s + 1), 1000);    // updater avoids stale state
  return () => clearInterval(id);
}, []);

// 4) WebSocket / SignalR connection that depends on a prop
useEffect(() => {
  const conn = createConnection(serverUrl, roomId);
  conn.connect();
  return () => conn.disconnect();
}, [serverUrl, roomId]);    // reconnects when room changes
```

8.3 Fetching Data in an Effect (and Race Conditions)

✎ Definition

Loading data inside an effect, and preventing **race conditions** (an older, slower response overwriting a newer one) with an ignore flag or an AbortController in the cleanup.

```
function TxnDetails({ txnId }) {
  const [txn, setTxn] = useState(null);
  const [status, setStatus] = useState('loading');    // 'loading' | 'success' | 'error'

  useEffect(() => {
    const controller = new AbortController();
    setStatus('loading');
    fetch(`/api/txns/${txnId}`, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(res.statusText); return res.json(); })
      .then(data => { setTxn(data); setStatus('success'); })
      .catch(err => { if (err.name !== 'AbortError') setStatus('error'); });
    return () => controller.abort();         // cancel the stale request
  }, [txnId]);

  if (status === 'loading') return <Spinner />;
  if (status === 'error') return <p>Failed to load</p>;
  return <TxnCard txn={txn} />;
}
```

⚡ Tricky Point

Race condition: user clicks txn 1 then quickly txn 2. If response 1 arrives *after* response 2, the screen shows txn 1's data for txn 2. The cleanup (abort, or an `ignore` flag) fixes it. This is a classic interview scenario.

```
useEffect(() => {
  let ignore = false;
  getTxn(txnId).then(data => { if (!ignore) setTxn(data); });
  return () => { ignore = true; };
}, [txnId]);
```

i Note

The effect callback **cannot be async** directly (it must return a cleanup function or nothing, not a Promise). Define an async function inside and call it. In real apps prefer a data library (TanStack Query, RTK Query, SWR) or framework loaders - they handle caching, deduplication, retries and race conditions.

8.4 Dependency Array Pitfalls

✎ Definition

Common mistakes with the dependency array: missing dependencies (stale values), objects or functions recreated on every render (infinite loops), and disabling the lint rule.

1. Missing dependencies (stale values)

```
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);   // 'count' is always 0 here
  return () => clearInterval(id);
}, []);                          // count stuck at 1!  Fix: setCount(c => c + 1)
```

Always include every reactive value (props, state, values derived from them) used inside the effect. The `eslint-plugin-react-hooks` rule `exhaustive-deps` checks this - don't silence it.

2. Infinite loops

```
useEffect(() => { setCount(count + 1); });              // no array -> runs after every render -> loop
useEffect(() => { setData(fetchSomething()); }, [data]); // updates its own dependency -> loop
const options = { pageSize: 20 };                       // NEW object every render
useEffect(() => { load(options); }, [options]);         // runs after every render!
```

Fixes for object/function dependencies: move them **inside** the effect, move constants **outside** the component, depend on primitives (`options.pageSize`), or memoize with `useMemo`/`useCallback`.

⚡ Tricky Point

Dependencies are compared with `Object.is`. Objects, arrays and functions created during render are **new every render**, so an effect depending on them runs every time.

3. Effects run twice in development

StrictMode mounts, unmounts and re-mounts every component in development to check that your cleanup is correct. If double-running breaks something (two analytics events, two POSTs), your effect is missing a cleanup or should be an event handler instead.

8.5 You Might Not Need an Effect

> **✎ Definition**
>
> Many effects are unnecessary. Derived data should be computed during render, and responses to user actions belong in event handlers rather than in effects.

Effects are an **escape hatch**. Many effects in real codebases are unnecessary and cause extra renders and bugs.

Situation	Don't (effect)	Do instead
Derived data	`useEffect(() => setFull(first + ' ' + last), [first, last])`	`const full = first + ' ' + last;` during render
Expensive calculation	Effect + state	`useMemo(() => filter(list), [list])`
Reset state on prop change	Effect that calls `setX` when id changes	`<Profile key={userId} />`
Respond to a user action (submit, buy)	Effect watching a flag	Do it directly in the event handler
Notify parent of change	Effect calling `onChange(value)`	Call `onChange` in the same handler that sets state
Chains of effects setting state	Effect A sets B, effect B sets C	Compute everything in one handler
Subscribe to external store	Effect + setState	`useSyncExternalStore`

> **★ Interview Tip**
>
> Rule of thumb: **"Does this code run because the component was displayed, or because of a specific user interaction?"** Displayed -> effect. Interaction -> event handler.

8.6 useLayoutEffect

> **✎ Definition**
>
> `useLayoutEffect` works like useEffect but runs **synchronously after DOM changes and before the browser paints**. Use it to measure layout and avoid visual flicker.

Same signature as `useEffect` but runs **synchronously after DOM mutations and before the browser paints**. Use it to measure the DOM and adjust layout without flicker (tooltips, popovers positioning).

```
useLayoutEffect(() => {
  const { height } = tooltipRef.current.getBoundingClientRect();
  setTooltipHeight(height);          // re-render happens before paint, no flicker
}, []);
```

> **⚠ Watch Out**
>
> `useLayoutEffect` blocks painting - keep it fast. It doesn't run during server rendering. `useInsertionEffect` is a third variant intended only for CSS-in-JS libraries.

8.7 useEffectEvent (React 19.2)

> **✎ Definition**
>
> `useEffectEvent` (React 19.2) defines a function, called from an effect, that always sees the **latest** props and state without needing to be a dependency of the effect.

Sometimes an effect needs the **latest** value of something without re-running when it changes.

```
function ChatRoom({ roomId, theme }) {
  const onConnected = useEffectEvent(() => {
    showNotification('Connected!', theme);      // always reads the latest theme
  });
  useEffect(() => {
    const conn = createConnection(roomId);
    conn.on('connected', () => onConnected());
    conn.connect();
    return () => conn.disconnect();
  }, [roomId]);                                  // theme change does NOT reconnect
}
```

9. Hooks in Depth

9.1 Rules of Hooks

✎ Definition

The two **Rules of Hooks**: call hooks only at the **top level** (not inside conditions or loops), and only from React function components or custom hooks.

1. **Only call hooks at the top level** of a component or custom hook - never inside conditions, loops, nested functions, or after an early `return`. 2. **Only call hooks from React functions** (components or custom hooks), not regular JS functions or class components.

```
function Bad({ isAdmin }) {
  if (isAdmin) { const [x, setX] = useState(0); }  // WRONG: conditional hook
  if (!user) return null;
  const [y] = useState(0);                         // WRONG: after an early return
}
```

★ Interview Tip

"Why must hooks be called in the same order?" - React doesn't identify hooks by name. It stores each component's hook state in an ordered list (a linked list on the fiber) and matches them by **call order**. A conditional hook shifts the order and React would give state to the wrong hook. (The new `use()` API is the exception: it can be called conditionally.)

9.2 Hooks Overview

> ✎ **Definition**
>
> **Hooks** are functions starting with use that give function components React features: state, effects, context, refs, memoisation and more.

Hook	Purpose
`useState`	Local state
`useReducer`	Complex state with a reducer function
`useEffect` / `useLayoutEffect`	Side effects after render / before paint
`useRef`	Mutable value that doesn't trigger re-render; DOM references
`useContext`	Read a context value
`useMemo`	Cache a computed value
`useCallback`	Cache a function
`useId`	Unique IDs for accessibility attributes (SSR-safe)
`useTransition`	Mark updates as non-urgent; get `isPending`
`useDeferredValue`	Defer re-rendering a slow part of the UI
`useImperativeHandle`	Customise what a parent receives through a ref
`useSyncExternalStore`	Subscribe to an external store safely
`useDebugValue`	Label custom hooks in DevTools
`use` (19)	Read a Promise or context; can be called conditionally
`useActionState` (19)	State + pending for form actions
`useOptimistic` (19)	Show optimistic UI while an async action runs
`useFormStatus` (19, react-dom)	Pending status of the parent form
`useEffectEvent` (19.2)	Non-reactive event logic used inside effects

9.3 useRef

> ✎ **Definition**
>
> useRef returns a mutable `{ current }` object that persists across renders **without causing re-renders**. It is used for DOM access and for storing mutable values.

`useRef` returns a mutable object `{ current: value }` that **persists across renders** and **does not cause a re-render** when changed.

```
// 1) Access DOM nodes
function SearchBox() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);
  return <input ref={inputRef} />;
}

// 2) Store mutable values: timer ids, previous values, flags
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null);
  const start = () => { intervalRef.current = setInterval(() => setTime(t => t + 1), 1000); };
  const stop = () => clearInterval(intervalRef.current);
  return <><p>{time}s</p><button onClick={start}>Start</button><button onClick={stop}>Stop</button></>;
}

// 3) Keep the latest callback without re-running an effect
function useInterval(callback, delay) {
  const savedCallback = useRef(callback);
  useEffect(() => { savedCallback.current = callback; });     // update after each render
  useEffect(() => {
    const id = setInterval(() => savedCallback.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}
```

useState	useRef
Changing it re-renders	Changing `.current` does not re-render
Value is a snapshot per render	Always the latest, mutable value
For data shown in the UI	For data not shown: ids, DOM nodes, instances

> ⚡ **Tricky Point**
>
> Don't read or write `ref.current` **during rendering** (except lazy initialisation) - it makes rendering unpredictable. Use refs in event handlers and effects. Also, `ref.current` is `null` during the first render because the DOM node doesn't exist yet.

Refs to your own components: forwardRef vs ref as a prop

```
// React 19: ref is a regular prop for function components
function TextInput({ ref, ...props }) { return <input ref={ref} {...props} />; }

// React 18 and earlier: needed forwardRef
const TextInput18 = forwardRef((props, ref) => <input ref={ref} {...props} />);

// Expose a limited API with useImperativeHandle
function VideoPlayer({ ref, src }) {
  const videoRef = useRef(null);
  useImperativeHandle(ref, () => ({
    play: () => videoRef.current.play(),
    pause: () => videoRef.current.pause(),
  }));
  return <video ref={videoRef} src={src} />;
}
```

9.4 useReducer

> **✎ Definition**
>
> `useReducer` manages complex state with a **reducer** function `(state, action) => newState` and a `dispatch` function. It suits state with many related transitions.

Good when state has **many sub-values**, the next state depends on the previous one in complex ways, or many event handlers update the same state. The logic moves into a pure **reducer** function that is easy to test.

```
const initialState = { items: [], status: 'idle', error: null };

function cartReducer(state, action) {
  switch (action.type) {
    case 'added':
      return { ...state, items: [...state.items, action.item] };
    case 'removed':
      return { ...state, items: state.items.filter(i => i.id !== action.id) };
    case 'qtyChanged':
      return { ...state, items: state.items.map(i =>
        i.id === action.id ? { ...i, qty: action.qty } : i) };
    case 'cleared':
      return initialState;
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function Cart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);
  return (
    <>
      <button onClick={() => dispatch({ type: 'added', item: { id: 1, name: 'Pen', qty: 1 } })}>Add</button>
      <button onClick={() => dispatch({ type: 'cleared' })}>Clear</button>
      <p>{state.items.length} items</p>
    </>
  );
}
```

> **⚡ Tricky Point**
>
> Reducers must be **pure**: no API calls, no mutations, no random values. They also run twice in StrictMode. `dispatch` has a stable identity - safe to omit from dependency arrays and pass to children.

9.5 useContext

> **✎ Definition**
>
> `useContext` reads the current value of a React **context** from the nearest Provider above, without passing props through every level.

See Chapter 10 for a full explanation. Quick form:

```
const ThemeContext = createContext('light');
function Button() {
  const theme = useContext(ThemeContext);     // nearest Provider above, or the default value
  return <button className={`btn-${theme}`}>OK</button>;
}
```

9.6 useMemo

> **✎ Definition**
>
> `useMemo` **caches the result** of an expensive calculation between renders and recomputes it only when its dependencies change.

Caches the **result of a calculation** between renders and recalculates only when dependencies change.

```
function TxnReport({ txns, query }) {
  const visible = useMemo(
    () => txns.filter(t => t.id.includes(query)).sort((a, b) => b.amount - a.amount),
    [txns, query]
  );
  const chartOptions = useMemo(() => ({ type: 'bar', stacked: true }), []);  // stable object for memo child
  return <Chart data={visible} options={chartOptions} />;
}
```

9.7 useCallback

> **✎ Definition**
>
> `useCallback` **caches a function definition** between renders, so a memoised child receives the same function reference.

Caches a **function definition** between renders. `useCallback(fn, deps)` equals `useMemo(() => fn, deps)`.

```
const MemoRow = memo(function Row({ item, onDelete }) { /* ... */ });

function List({ items }) {
  const [selected, setSelected] = useState(null);
  const handleDelete = useCallback(id => {
    api.delete(id);
  }, []);                                    // same function every render
  return items.map(i => <MemoRow key={i.id} item={i} onDelete={handleDelete} />);
}
```

> **⚡ Tricky Point**
>
> `useMemo`/`useCallback` **only help** when (a) the calculation is genuinely expensive, or (b) the value is passed to a `memo` component or used as a dependency of another hook. Wrapping everything adds overhead and noise. With the **React Compiler** (2025), most manual memoization becomes unnecessary because the compiler memoizes automatically.

> **★ Interview Tip**
>
> "useMemo vs useCallback?" - `useMemo` caches a computed **value**; `useCallback` caches a **function** itself. Both re-compute only when dependencies change, and both mainly exist to keep references stable for memoized children and effect dependencies.

9.8 useId

> **✎ Definition**
>
> useId generates a **unique, stable ID** that matches between server and client, for accessibility attributes such as `htmlFor` and `aria-describedby`.

```
function EmailField() {
  const id = useId();                        // e.g. ':r1:' - unique, stable, same on server & client
  return (<><label htmlFor={id}>Email</label><input id={id} aria-describedby={`${id}-hint`} /></>);
}
```

> **⚠ Watch Out**
>
> Don't use `useId` for list keys - keys should come from your data.

9.9 useTransition and useDeferredValue (Concurrent Features)

> **✎ Definition**
>
> **Concurrent features**: `useTransition` marks a state update as non-urgent so the UI stays responsive, and `useDeferredValue` defers re-rendering with a value that lags behind.

Some updates are **urgent** (typing in an input) and some can wait (re-rendering a 10,000-row filtered table). Concurrent rendering lets React interrupt slow non-urgent renders to keep the UI responsive.

```
function Search({ allTxns }) {
  const [query, setQuery] = useState('');
  const [filter, setFilter] = useState('');
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    setQuery(e.target.value);                          // urgent: update the input now
    startTransition(() => setFilter(e.target.value)); // non-urgent: can be interrupted
  }
  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <small>Updating...</small>}
      <BigTable rows={allTxns} filter={filter} />
    </>
  );
}

function Search2({ query }) {
  const deferredQuery = useDeferredValue(query);      // lags behind during heavy renders
  const isStale = query !== deferredQuery;
  return <div style={{ opacity: isStale ? 0.5 : 1 }}><SlowList query={deferredQuery} /></div>;
}
```

useTransition	useDeferredValue
You control the state update (wrap the setter)	You only receive a value (e.g. a prop)
Gives `isPending`	Compare value vs deferred value
Can wrap async functions in React 19 (Actions)	Optional initial value (React 19)

> **⚡ Tricky Point**
>
> Transitions are not debouncing: no fixed delay. React starts rendering immediately but can abandon the work if a newer urgent update arrives. Inputs themselves must **not** be updated inside a transition, or typing becomes laggy.

9.10 useSyncExternalStore

> **✎ Definition**
>
> `useSyncExternalStore` subscribes a component to an **external store** (outside React) in a way that is safe with concurrent rendering.

The safe way to subscribe to stores outside React (browser APIs, Redux/Zustand internals) without "tearing" in concurrent mode.

```
function useOnlineStatus() {
  return useSyncExternalStore(
    cb => { window.addEventListener('online', cb); window.addEventListener('offline', cb);
            return () => { window.removeEventListener('online', cb); window.removeEventListener('offline', cb); }; },
    () => navigator.onLine,     // client snapshot
    () => true                  // server snapshot
  );
}
```

9.11 React 19 Hooks: use, useOptimistic

> **✎ Definition**
>
> New React 19 hooks: `use` reads a promise or context (and can be called conditionally), and `useOptimistic` shows an optimistic UI value while an async action is in progress.

```
// use(): read a promise (suspends until resolved) or a context, even conditionally
function Comments({ commentsPromise }) {
  const comments = use(commentsPromise);          // must be wrapped in <Suspense>
  return comments.map(c => <p key={c.id}>{c.text}</p>);
}
function Heading({ show }) {
  if (show) { const theme = use(ThemeContext); return <h1 className={theme}>Hi</h1>; }   // allowed
  return null;
}
// useOptimistic(): show the expected result immediately, roll back on failure
function Likes({ count, onLike }) {
  const [optimisticCount, addOptimistic] = useOptimistic(count, (cur, delta) => cur + delta);
  async function like() {
    addOptimistic(1);            // UI shows +1 instantly
    await onLike();              // server confirms; parent passes new count
  }
  return <form action={like}><button>Like {optimisticCount}</button></form>;
}
```

> **⚠ Watch Out**
>
> Don't create the promise passed to `use()` inside the same component's render (a new promise each render = infinite suspense). Create it in a Server Component, a loader, or cache it.

9.12 Custom Hooks

> **✎ Definition**
>
> A **custom hook** is a function starting with use that combines other hooks to share **stateful logic** (not state itself) between components.

A custom hook is a function starting with **use** that calls other hooks. It lets you **reuse stateful logic** (not state itself - each call gets its own independent state).

```
function useFetch(url) {
  const [state, setState] = useState({ data: null, error: null, loading: true });
  useEffect(() => {
    const ctrl = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error(r.statusText); return r.json(); })
      .then(data => setState({ data, error: null, loading: false }))
      .catch(error => { if (error.name !== 'AbortError') setState({ data: null, error, loading: false }); });
    return () => ctrl.abort();
  }, [url]);
  return state;
}

function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);                  // restart timer on every keystroke
  }, [value, delay]);
  return debounced;
}

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try { return JSON.parse(localStorage.getItem(key)) ?? initial; } catch { return initial; }
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}
```

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(o => !o), []);
  return [on, toggle];
}

// Usage
function TxnSearch() {
  const [query, setQuery] = useState('');
  const debounced = useDebounce(query, 400);
  const { data, loading } = useFetch(`/api/txns?q=${encodeURIComponent(debounced)}`);
  const [compact, toggleCompact] = useToggle();
  return (/* ... */ null);
}
```

> **⚡ Tricky Point**
>
> Two components using `useToggle()` get **separate** state. To *share* state, lift it up or use context/a store. The `use` prefix matters: it tells React and the linter to apply the Rules of Hooks.

10. Context API and State Management

10.1 The Problem: Prop Drilling

> ✎ **Definition**
>
> **Prop drilling** is passing props through many intermediate components that don't use them, only to reach a deeply nested component that does.

Prop drilling is passing props through many intermediate components that don't use them, just to reach a deep child.

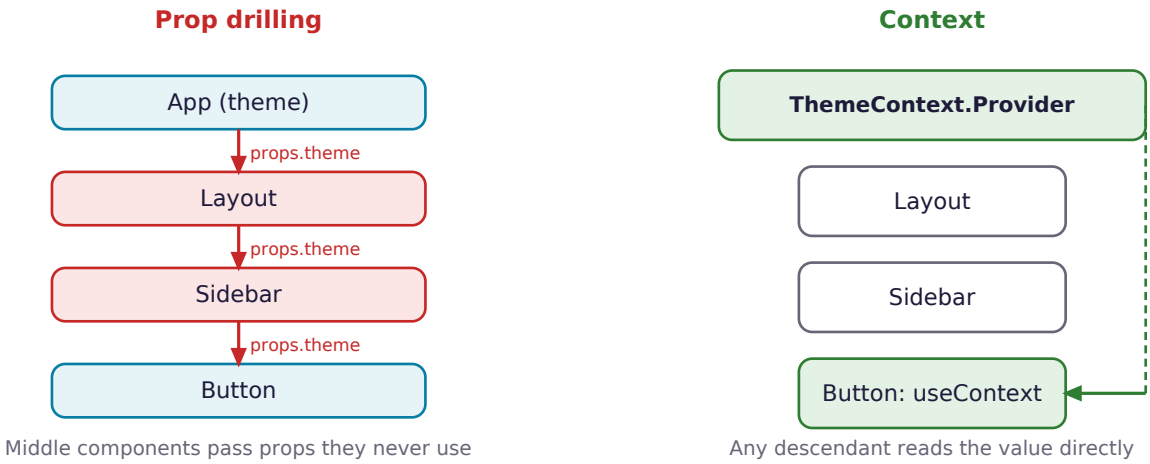


Figure 10.1 - Prop drilling vs Context

10.2 Context API Step by Step

✎ Definition

The **Context API** shares a value with a whole subtree: create it with `createContext`, provide it with a Provider, and read it with `useContext` at any depth.

```
// 1) Create
import { createContext, useContext, useState, useMemo } from 'react';
const AuthContext = createContext(null);

// 2) Provide (React 19 also allows <AuthContext value={...}> without .Provider)
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = async (creds) => setUser(await api.login(creds));
  const logout = () => setUser(null);
  const value = useMemo(() => ({ user, login, logout }), [user]);   // stable object
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// 3) Consume through a custom hook (nice error if used outside the provider)
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
// 4) Use anywhere below
function Navbar() {
  const { user, logout } = useAuth();
  return user ? <button onClick={logout}>Logout {user.name}</button> : <a href="/login">Login</a>;
}
// main.jsx
<AuthProvider><App /></AuthProvider>
```

Good uses of context: current user/auth, theme, locale/language, feature flags, a router, a form library's internal state - values that are **global-ish** and **change rarely**.

⚡ Tricky Point

Every consumer re-renders when the context value changes, and `memo` does not stop it. If the provider passes `value={{ user, login }}` without `useMemo`, a new object is created on every provider render and **all consumers re-render** every time. Memoize the value, and split frequently changing data into separate contexts (e.g. `StateContext` and `DispatchContext`).

⚡ Tricky Point

The default value in `createContext(default)` is used **only** when there is no provider above. It is not a fallback for `undefined` values passed by a provider.

Context + useReducer: a mini Redux

```
const TasksContext = createContext(null);
const TasksDispatchContext = createContext(null);

export function TasksProvider({ children }) {
  const [tasks, dispatch] = useReducer(tasksReducer, []);
  return (
    <TasksContext.Provider value={tasks}>
      <TasksDispatchContext.Provider value={dispatch}>   {/* dispatch never changes */}
        {children}
      </TasksDispatchContext.Provider>
    </TasksContext.Provider>
  );
}
```

★ Interview Tip

"Is Context a state management library?" - No. Context is a **dependency-injection / transport** mechanism: it passes a value down the tree. The state itself still lives in `useState`/`useReducer`. It has no selectors, so it's not ideal for large, frequently-updated global state.

10.3 Types of State

✎ Definition

Categories of state: **local UI** state, **shared/global client** state, **server** state (cached API data), **URL** state and **form** state. Each has its own best tool.

Kind	Example	Best tool
Local UI state	Modal open, input text, active tab	`useState`, `useReducer`
Shared client state	Cart, theme, auth user, wizard steps	Context, Zustand, Redux Toolkit, Jotai
Server state (cached remote data)	Transactions list, user profile from API	TanStack Query, RTK Query, SWR
URL state	Page number, filters, selected tab	React Router search params
Form state	Field values, errors, touched	React Hook Form, `useActionState`

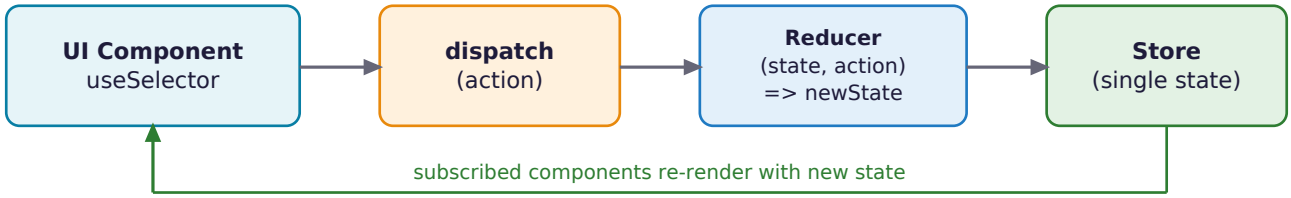
★ Interview Tip

Most "global state" in typical apps is actually **server state**. Moving it to TanStack Query/RTK Query usually eliminates most Redux boilerplate. Keep only truly client-side shared data in a global store.

10.4 Redux Toolkit (RTK)

> ✎ **Definition**
>
> **Redux Toolkit** is the official, simplified way to use Redux: a central store with `createSlice` for reducers and actions, Immer for immutable updates, and RTK Query for data fetching.

Redux keeps app state in **one store**, changed only by dispatching **actions** handled by pure **reducers**. Redux Toolkit is the official, modern way to write Redux (no hand-written action types, uses Immer).



One-way flow: View → Action → Reducer → Store → View. Reducers are pure; side effects go in thunks/RTK Query.

Figure 10.2 - Redux data flow

```
// features/txns/txnSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

export const fetchTxns = createAsyncThunk('txns/fetch', async (bankId) => {
  const res = await fetch(`/api/banks/${bankId}/txns`);
  return res.json();
});

const txnSlice = createSlice({
  name: 'txns',
  initialState: { items: [], status: 'idle', error: null, filter: 'ALL' },
  reducers: {
    filterChanged(state, action) { state.filter = action.payload; },   // "mutation" is OK (Immer)
    txnRemoved(state, action) { state.items = state.items.filter(t => t.id !== action.payload); },
  },
  extraReducers: builder => {
    builder
      .addCase(fetchTxns.pending, state => { state.status = 'loading'; })
      .addCase(fetchTxns.fulfilled, (state, action) => { state.status = 'succeeded'; state.items = action.payload; })
      .addCase(fetchTxns.rejected, (state, action) => { state.status = 'failed'; state.error = action.error.message; });
  },
});
export const { filterChanged, txnRemoved } = txnSlice.actions;
export default txnSlice.reducer;

// app/store.js
export const store = configureStore({ reducer: { txns: txnReducer } });

// main.jsx
<Provider store={store}><App /></Provider>
// component
function TxnList() {
  const dispatch = useDispatch();
  const items = useSelector(s => s.txns.items);          // re-renders only when this slice changes
  const status = useSelector(s => s.txns.status);
  useEffect(() => { if (status === 'idle') dispatch(fetchTxns('HDFC')); }, [status, dispatch]);
  return items.map(t => <TxnRow key={t.id} {...t} />);
}
```

> ⚡ **Tricky Point**
>
> A selector that returns a **new object/array every time** (`useSelector(s => s.txns.items.filter(...))`) makes the component re-render on **every** store update. Use memoized selectors (`createSelector` from Reselect/RTK) or select primitives.

RTK Query

```
export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Txn'],
  endpoints: b => ({
    getTxns: b.query({ query: bankId => `banks/${bankId}/txns`, providesTags: ['Txn'] }),
    reverseTxn: b.mutation({ query: id => ({ url: `txns/${id}/reverse`, method: 'POST' }), invalidatesTags: ['Txn'] }),
  }),
});
export const { useGetTxnsQuery, useReverseTxnMutation } = api;
const { data, isLoading, error } = useGetTxnsQuery('HDFC');
```

10.5 Zustand (Lightweight Alternative)

✎ Definition

Zustand is a small, hook-based state management library. You create a store with state and actions and use it in any component without Providers.

```
import { create } from 'zustand';
const useCartStore = create(set => ({
  items: [],
  add: item => set(s => ({ items: [...s.items, item] })),
  clear: () => set({ items: [] }),
}));
function CartBadge() {
  const count = useCartStore(s => s.items.length);    // subscribe to a slice only
  return <span>{count}</span>;
}
```

10.6 Choosing a Tool

✎ Definition

A guide to picking a state tool: local state first, Context for low-frequency shared values, Zustand or Redux for complex global state, and TanStack Query for server data.

Tool	Strengths	Trade-offs
useState / useReducer	Built-in, simple	Local only
Context	Built-in, avoids drilling	All consumers re-render; no selectors
Redux Toolkit	Predictable, DevTools time-travel, middleware, large teams	More concepts and setup
Zustand	Tiny, minimal API, selectors, no provider	Less structure for very large apps
Jotai / Recoil	Atomic fine-grained state	Different mental model
TanStack Query	Caching, refetching, retries for server data	Not for client-only state

11. Routing with React Router

11.1 Client-Side Routing

> ✎ **Definition**
>
> **Client-side routing** changes the URL and the displayed view **in the browser**, using the History API, without a full page reload from the server.

React Router maps **URLs to components** without full page reloads, using the browser History API. Version 6.4+ added data APIs (loaders/actions); v7 merged Remix and adds an optional framework mode. The examples use the widely used library ("declarative/data") mode.

11.2 Basic Setup

> ✎ **Definition**
>
> Setting up **React Router**: wrap the app in a router (`createBrowserRouter` or `BrowserRouter`) and map URL paths to components with routes.

```
import { BrowserRouter, Routes, Route, Link, NavLink } from 'react-router';   // v7 ('react-router-dom' in v6)

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <NavLink to="/txns" className={({ isActive }) => isActive ? 'active' : ''}>Transactions</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/txns" element={<TxnList />} />
        <Route path="/txns/:txnId" element={<TxnDetails />} />      {/* URL param */}
        <Route path="/login" element={<Login />} />
        <Route path="*" element={<NotFound />} />                     {/* 404 */}
      </Routes>
    </BrowserRouter>
  );
}
```

> ⚡ **Tricky Point**
>
> Use `<Link>`/`<NavLink>`, not `<a href>`, for internal navigation. A plain `<a>` triggers a full page reload, destroying all React state and re-downloading the app.

11.3 Params, Search Params and Navigation

✎ Definition

Reading dynamic URL segments (`useParams`), query strings (`useSearchParams`), and navigating with `<Link>`, `<NavLink>` or `useNavigate`.

```
import { useParams, useSearchParams, useNavigate, useLocation } from 'react-router';

function TxnDetails() {
  const { txnId } = useParams();                       // /txns/T100 -> 'T100' (always a string)
  const navigate = useNavigate();
  return <button onClick={() => navigate('/txns', { replace: true })}>Back</button>;
}

function TxnList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const page = Number(searchParams.get('page') ?? 1);   // /txns?page=2&status=FAILED
  const status = searchParams.get('status') ?? 'ALL';
  return <Pager page={page} onChange={p => setSearchParams({ page: p, status })} />;
}
navigate(-1);                                          // go back
navigate('/login', { state: { from: location.pathname } });   // pass state
```

★ Interview Tip

Store filters, sorting and pagination in **search params** instead of component state: the view becomes shareable, bookmarkable, and survives refresh and back/forward navigation.

11.4 Nested Routes and Layouts

✎ Definition

Nested routes render child routes inside a parent **layout** component at the `<Outlet />` position, so shared UI like headers and sidebars stays in place.

```
<Routes>
  <Route path="/" element={<AppLayout />}>           {/* shared header/sidebar */}
    <Route index element={<Dashboard />} />           {/* matches "/" exactly */}
    <Route path="reports" element={<Reports />}>
      <Route path=":reportId" element={<ReportView />} />
    </Route>
    <Route path="settings/*" element={<Settings />} />
  </Route>
</Routes>
function AppLayout() {
  return (
    <div className="layout">
      <Sidebar />
      <main><Outlet /></main>                         {/* child route renders here */}
    </div>
  );
}
```

11.5 Protected Routes

> ✎ **Definition**
>
> **Protected routes** allow access only when a condition is met (for example, the user is logged in) and otherwise redirect, such as to the login page.

```
function RequireAuth({ children, roles }) {
  const { user } = useAuth();
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/forbidden" replace />;
  return children;
}
<Route path="/admin" element={<RequireAuth roles={['ADMIN']}><AdminPage /></RequireAuth>} />
```

> ⚠ **Watch Out**
>
> Client-side route guards are a **UX feature, not security**. Anyone can modify JS in the browser. The API must always verify the JWT/session and roles on the server.

11.6 Data Routers: Loaders and Actions

> ✎ **Definition**
>
> In React Router data routers, a **loader** fetches data before a route renders (`useLoaderData`) and an **action** handles form submissions for that route.

```
const router = createBrowserRouter([
  {
    path: '/txns/:txnId',
    element: <TxnDetails />,
    loader: async ({ params }) => fetch(`/api/txns/${params.txnId}`),   // runs BEFORE render
    action: async ({ request, params }) => {
      const form = await request.formData();
      await api.update(params.txnId, Object.fromEntries(form));
      return redirect('/txns');
    },
    errorElement: <RouteError />,
  },
]);
createRoot(root).render(<RouterProvider router={router} />);

function TxnDetails() {
  const txn = useLoaderData();                 // data already loaded - no loading-state effect
  const navigation = useNavigation();          // navigation.state: 'idle' | 'loading' | 'submitting'
  return <Form method="post"><input name="note" defaultValue={txn.note} /><button>Save</button></Form>;
}
```

11.7 Lazy-Loaded Routes

> ✎ **Definition**
>
> Loading a route's code only when the user visits it, using `React.lazy` or the route `lazy` property, to make the initial bundle smaller.

```
const Reports = lazy(() => import('./pages/Reports'));
<Route path="/reports" element={<Suspense fallback={<Spinner />}><Reports /></Suspense>} />
```

> **i Note**
>
> Deploying an SPA with `BrowserRouter`: the server must return `index.html` for unknown paths (fallback / rewrite rule in IIS, Nginx, Azure Static Web Apps). Otherwise refreshing `/txns/T1` gives a 404. `HashRouter` (`/#/txns`) avoids this but produces ugly URLs.

12. Data Fetching and APIs

12.1 Approaches Compared

✎ Definition

A comparison of ways to load data: fetch in an effect, a library such as TanStack Query or SWR, router loaders, and Server Components, with the trade-offs of each.

Approach	When to use
`fetch` / axios in `useEffect`	Learning, tiny apps; you must handle loading, errors, races, caching yourself
Custom `useFetch` hook	Small apps; still no cache/dedupe
TanStack Query / SWR / RTK Query	Most production SPAs: caching, background refetch, retries, pagination, mutations
Router loaders (React Router, Next.js)	Fetch per route before render; avoids waterfalls
Server Components (Next.js)	Fetch on the server, send ready HTML; no client JS for data

12.2 axios with Interceptors (JWT Example)

✎ Definition

axios is an HTTP client library. **Interceptors** run on every request or response, for example to attach a JWT or to refresh an expired token and retry.

```
import axios from 'axios';
export const api = axios.create({ baseURL: import.meta.env.VITE_API_URL, timeout: 15000 });

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
api.interceptors.response.use(
  res => res,
  async err => {
    if (err.response?.status === 401) { window.location.href = '/login'; }
    return Promise.reject(err);
  }
);
const { data } = await api.get('/txns', { params: { page: 1, status: 'FAILED' } });
await api.post('/txns/reconcile', { date: '2026-10-01' });
```

⚠ Watch Out

Environment variables in a Vite/CRA bundle (`VITE_*`, `REACT_APP_*`) are **embedded in public JavaScript**. Never put secrets (API keys, DB passwords) there. Storing JWTs in `localStorage` is exposed to XSS; HttpOnly cookies are safer for sensitive apps.

12.3 TanStack Query (React Query)

> ✎ **Definition**
>
> **TanStack Query** is a server-state library that handles fetching, **caching**, deduplication, background refetching, retries and mutations through hooks such as useQuery.

TanStack Query manages **server state**: it caches responses by a query key, deduplicates identical requests, refetches stale data in the background, retries failures and synchronises mutations.

```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
const queryClient = new QueryClient({ defaultOptions: { queries: { staleTime: 60_000 } } });
<QueryClientProvider client={queryClient}><App /></QueryClientProvider>

function TxnList({ bankId, page }) {
  const { data, isPending, isError, error, isFetching } = useQuery({
    queryKey: ['txns', bankId, page],            // cache key: changes -> new fetch
    queryFn: ({ signal }) => api.get(`/banks/${bankId}/txns`, { params: { page }, signal }).then(r => r.data),
    placeholderData: prev => prev,               // keep old page visible while loading next
    enabled: !!bankId,                           // dependent query
  });
  if (isPending) return <Spinner />;
  if (isError) return <ErrorBox error={error} />;
  return <><TxnTable rows={data.items} />{isFetching && <small>Refreshing...</small>}</>;
}

function ReverseButton({ txnId }) {
  const qc = useQueryClient();
  const mutation = useMutation({
    mutationFn: () => api.post(`/txns/${txnId}/reverse`),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['txns'] }),   // refetch lists
  });
  return <button disabled={mutation.isPending} onClick={() => mutation.mutate()}>Reverse</button>;
}
```

Concept	Meaning
`staleTime`	How long data is considered fresh (no refetch). Default 0.
`gcTime`	How long unused cache stays in memory (default 5 min).
`isPending` vs `isFetching`	No data yet vs any request in flight (including background).
Invalidation	Mark cached queries stale so they refetch.
Optimistic updates	Update the cache in `onMutate`, roll back in `onError`.

12.4 Loading, Error and Empty States

> ✎ **Definition**
>
> Handling every UI state of a request (loading, error, empty and success) so the user always sees meaningful feedback.

Every data view should handle **four** states: loading, error, empty and success. Prefer a single `status` value over several booleans, and show skeletons for better perceived performance.

12.5 Suspense for Data

> ✎ **Definition**
>
> **Suspense** lets a component "wait" for data. React shows the nearest <Suspense fallback> until the data is ready, declaratively.

`<Suspense>` shows a fallback while children are "suspended" (waiting for lazy code or data from a Suspense-enabled source: `use()`, `useSuspenseQuery`, Next.js server components).

```
<Suspense fallback={<TableSkeleton />}>
  <TxnTable />                  {/* uses useSuspenseQuery inside: data is never undefined */}
</Suspense>
```

> ⚡ **Tricky Point**
>
> Suspense does **not** detect a plain `fetch` in `useEffect`. The data source must integrate with Suspense (throw/return a pending promise through `use()` or a library).

12.6 Avoiding Waterfalls

> ✎ **Definition**
>
> A **waterfall** is a chain of requests that run one after another because each one waits for the previous render. Avoid it by fetching in parallel, prefetching or hoisting requests.

```
// Waterfall: child fetch starts only after parent fetch finishes and renders
// Better: start requests in parallel at the route/loader level
const [user, txns] = await Promise.all([getUser(id), getTxns(id)]);
// or with TanStack Query: useQueries({ queries: [...] })
```

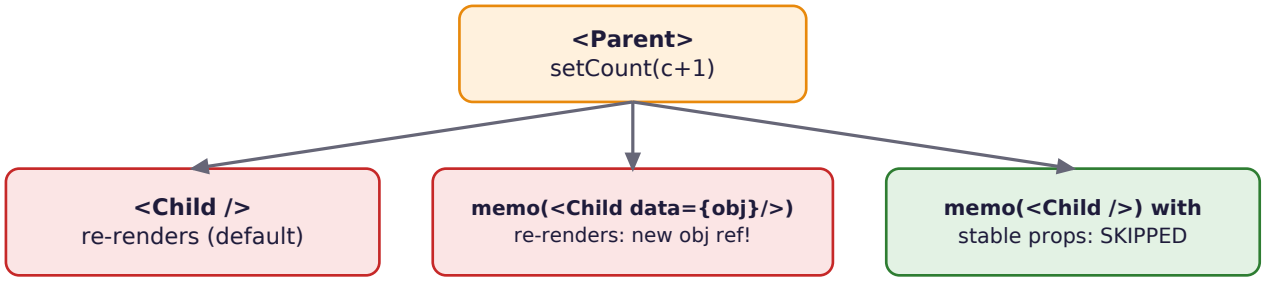
13. Performance Optimization

> **i Note**
>
> Golden rule: **measure first**. Use the React DevTools **Profiler** ("Highlight updates when components render") and the browser Performance tab. Most re-renders are cheap; optimise the ones that are slow and frequent.

13.1 React.memo

> **✎ Definition**
>
> `React.memo` wraps a component so it **skips re-rendering** when its props are shallowly equal to the previous props.

`memo` makes a component skip re-rendering when its props are **shallowly equal** to the previous props.



When a parent renders, ALL children render unless memoized AND props are shallow-equal.

Figure 13.1 - How memo interacts with parent re-renders

```
const TxnRow = memo(function TxnRow({ txn, onSelect }) {
  return <tr onClick={() => onSelect(txn.id)}><td>{txn.id}</td><td>{txn.amount}</td></tr>;
});

function TxnTable({ txns }) {
  const [selected, setSelected] = useState(null);
  const handleSelect = useCallback(id => setSelected(id), []);   // stable reference
  return txns.map(t => <TxnRow key={t.id} txn={t} onSelect={handleSelect} />);
}
const Chart = memo(ChartImpl, (prev, next) => prev.data.length === next.data.length);  // custom compare
```

> **⚡ Tricky Point**
>
> `memo` is **useless** if you pass new objects/arrays/functions every render: `<Row style={{ color: 'red' }} onClick={() => ...} />`. It also doesn't prevent re-renders caused by the component's **own state** or **context** changes. And `children` passed as JSX is a new object every render, which breaks memo too.

13.2 Avoiding Unnecessary Work

> ✎ **Definition**
>
> Techniques to reduce wasted renders: keep state local, move state down or lift content up, split components, and use stable props, keys and memoisation where measured.

- **Move state down:** put frequently changing state in the smallest component that needs it.
- **Lift content up:** pass expensive subtrees as `children` (see 7.3).
- Keep context values stable and split contexts by update frequency.
- Compute derived data during render with `useMemo` only when it's expensive.
- Avoid inline object/array literals as props to memoized children.
- Use stable `key`s; never random keys.

13.3 Code Splitting and Lazy Loading

> ✎ **Definition**
>
> **Code splitting** divides the bundle into smaller chunks that load on demand (`React.lazy` + `import()` + Suspense), so the app starts faster.

Ship less JavaScript up front by splitting the bundle and loading parts on demand.

```
import { lazy, Suspense } from 'react';
const ReportsPage = lazy(() => import('./pages/ReportsPage'));     // separate chunk
const HeavyChart = lazy(() => import('./components/HeavyChart'));

function App() {
  return (
    <Suspense fallback={<PageSpinner />}>
      <Routes>
        <Route path="/reports" element={<ReportsPage />} />
      </Routes>
    </Suspense>
  );
}
// Preload on hover for snappy navigation
const preload = () => import('./pages/ReportsPage');
<Link to="/reports" onMouseEnter={preload}>Reports</Link>
```

> ⚡ **Tricky Point**
>
> `lazy()` must be called **at module level**, not inside a component (otherwise state resets on every render). The imported module must have a **default export** (or map it: `lazy(() => import('./x').then(m => ({ default: m.Named })))`).

13.4 Large Lists: Virtualization

> ✎ **Definition**
>
> **Virtualization** (windowing) renders only the list rows that are visible on screen, so lists with thousands of items stay fast. Libraries: react-window, TanStack Virtual.

Rendering 50,000 table rows creates 50,000+ DOM nodes. **Virtualization (windowing)** renders only the visible rows.

```
import { useVirtualizer } from '@tanstack/react-virtual';
function BigList({ rows }) {
  const parentRef = useRef(null);
  const v = useVirtualizer({ count: rows.length, getScrollElement: () => parentRef.current, estimateSize: () => 36 });
  return (
    <div ref={parentRef} style={{ height: 600, overflow: 'auto' }}>
      <div style={{ height: v.getTotalSize(), position: 'relative' }}>
        {v.getVirtualItems().map(item => (
          <div key={item.key} style={{ position: 'absolute', top: item.start, height: 36, width: '100%' }}>
            {rows[item.index].id}
          </div>
        ))}
      </div>
    </div>
  );
}
```

Alternatives: react-window, AG Grid / MUI DataGrid (built-in virtualization), or server-side pagination.

13.5 Debouncing and Throttling

> ✎ **Definition**
>
> **Debouncing** runs a function only after the user stops triggering it for a delay (search boxes). **Throttling** runs it at most once per interval (scroll, resize).

```
const [query, setQuery] = useState('');
const debouncedQuery = useDebounce(query, 400);      // API called 400ms after typing stops
useQuery({ queryKey: ['search', debouncedQuery], queryFn: () => search(debouncedQuery) });
```

Debounce = wait until activity stops (search boxes). Throttle = at most once per interval (scroll, resize).

13.6 React Compiler

> ✎ **Definition**
>
> The **React Compiler** is a build-time tool that automatically memoises components and values, so manual useMemo, useCallback and React.memo are rarely needed.

The **React Compiler** (stable 1.0 in 2025) is a build-time Babel plugin that automatically memoizes components and values, making most manual `memo`, `useMemo` and `useCallback` unnecessary. It relies on your code following the Rules of React (pure render, no mutation of props/state).

13.7 Other Performance Checklist

> ✎ **Definition**
>
> Additional performance practices: production builds, bundle analysis, tree-shaking imports, image optimisation, and using transitions for heavy UIs.

- Production build (`npm run build`) - development mode is much slower (extra checks, StrictMode double renders).
- Analyse bundle size (`rollup-plugin-visualizer`, `source-map-explorer`); import only what you use (`import debounce from 'lodash/debounce'`).
- Optimise images (correct size, WebP/AVIF, `loading="lazy"`).
- Use `useTransition`/`useDeferredValue` for heavy filtering UIs.

- Use Web Workers for CPU-heavy work (parsing large Excel/CSV files).
- Avoid layout thrashing in effects (batch DOM reads and writes).

> **★ Interview Tip**
>
> "How would you optimise a slow React page?" - Profile to find what's slow; reduce unnecessary re-renders (state colocation, memo with stable props, split context); reduce work per render (useMemo, virtualization); reduce JS shipped (code splitting, lazy routes, tree-shaking); improve data loading (caching with React Query, parallel requests, pagination); and use concurrent features to keep input responsive.

14. Advanced Patterns

14.1 Higher-Order Components (HOC)

> ✎ **Definition**
>
> A **Higher-Order Component** is a function that takes a component and returns a new component with added behaviour, such as withAuth(Component). It is largely replaced by hooks.

A HOC is a **function that takes a component and returns a new, enhanced component**. Common in older code (`connect()` from react-redux, `withRouter`).

```
function withLoading(WrappedComponent) {
  return function WithLoading({ isLoading, ...rest }) {
    if (isLoading) return <Spinner />;
    return <WrappedComponent {...rest} />;
  };
}
const TxnTableWithLoading = withLoading(TxnTable);
<TxnTableWithLoading isLoading={loading} rows={rows} />

function withRole(Component, role) {
  return props => useAuth().user?.role === role ? <Component {...props} /> : <p>Access denied</p>;
}
```

> ⚡ **Tricky Point**
>
> Don't create HOCs inside render (`const X = withLoading(Table)` inside a component) - a new component type every render remounts the subtree. HOCs also cause "wrapper hell" and prop-name collisions; custom hooks usually replace them today.

14.2 Render Props

> ✎ **Definition**
>
> **Render props** is a pattern where a component receives a **function as a prop** and calls it to decide what to render, sharing its logic with the caller.

A component receives a **function as a prop** (often `children`) and calls it to decide what to render.

```
function MouseTracker({ children }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>{children(pos)}</div>;
}
<MouseTracker>{({ x, y }) => <p>Mouse at {x}, {y}</p>}</MouseTracker>
```

Today, the same logic would be a `useMousePosition()` custom hook. Render props remain useful in libraries (e.g. Formik `<Field>`, virtualized list item renderers).

14.3 Compound Components

> ✎ **Definition**
>
> **Compound components** are a set of components that work together and share implicit state through context, such as <Tabs><Tabs.List><Tabs.Panel>.

A set of components that work together and share implicit state via context - like `<select>` and `<option>`.

```
const TabsContext = createContext(null);
function Tabs({ defaultValue, children }) {
  const [active, setActive] = useState(defaultValue);
  return <TabsContext.Provider value={{ active, setActive }}>{children}</TabsContext.Provider>;
}
Tabs.List = function TabList({ children }) { return <div role="tablist">{children}</div>; };
Tabs.Tab = function Tab({ value, children }) {
  const { active, setActive } = useContext(TabsContext);
  return <button role="tab" aria-selected={active === value} onClick={() => setActive(value)}>{children}</button>;
};
```

```
Tabs.Panel = function Panel({ value, children }) {
  const { active } = useContext(TabsContext);
  return active === value ? <div role="tabpanel">{children}</div> : null;
};

<Tabs defaultValue="summary">
  <Tabs.List><Tabs.Tab value="summary">Summary</Tabs.Tab><Tabs.Tab value="details">Details</Tabs.Tab></Tabs.List>
  <Tabs.Panel value="summary"><Summary /></Tabs.Panel>
  <Tabs.Panel value="details"><Details /></Tabs.Panel>
</Tabs>
```

14.4 Controlled vs Uncontrolled Component APIs

> ✎ **Definition**
>
> A reusable component API can be **controlled** (the parent owns the state through value/onChange props) or **uncontrolled** (the component manages its own state with a default value), or support both.

Good reusable components support both: if a `value` prop is given, the parent controls it; otherwise the component manages its own state.

```
function Toggle({ checked: controlled, defaultChecked = false, onChange }) {
  const [internal, setInternal] = useState(defaultChecked);
  const isControlled = controlled !== undefined;
  const checked = isControlled ? controlled : internal;
  const toggle = () => { if (!isControlled) setInternal(!checked); onChange?.(!checked); };
  return <button aria-pressed={checked} onClick={toggle}>{checked ? 'ON' : 'OFF'}</button>;
}
```

14.5 Portals

> ✎ **Definition**
>
> A **Portal** (`createPortal`) renders children into a different DOM node, such as document.body, while keeping them in the same React tree. Used for modals and tooltips.

Render children into a **different DOM node** (outside the parent's DOM hierarchy) - for modals, tooltips, toasts that must escape `overflow: hidden` or z-index stacking.

```
import { createPortal } from 'react-dom';
function Modal({ open, onClose, children }) {
  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={onClose}>
      <div className="modal" role="dialog" aria-modal="true" onClick={e => e.stopPropagation()}>{children}</div>
    </div>,
    document.body
  );
}
```

⚡ Tricky Point

Even though a portal lives elsewhere in the DOM, it stays in the **React tree**: context works, and React events **bubble up through React parents**, not DOM parents. A click inside a portal modal triggers `onClick` handlers on the React ancestors of `<Modal>`.

14.6 Error Boundaries

✎ Definition

An **Error Boundary** is a class component that catches JavaScript errors in its child tree during rendering and shows a fallback UI instead of crashing the whole app.

An error boundary catches JavaScript errors **during rendering, in lifecycle methods and constructors** of its children, and shows a fallback UI instead of unmounting the whole app (a white screen).

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };
  static getDerivedStateFromError(error) { return { hasError: true, error }; }   // render fallback
  componentDidCatch(error, info) { logToServer(error, info.componentStack); }   // side effects
  render() {
    if (this.state.hasError) return this.props.fallback ?? <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
<ErrorBoundary fallback={<ErrorPage />}><Dashboard /></ErrorBoundary>
```

```
// Popular library: react-error-boundary  ->  <ErrorBoundary FallbackComponent={...} onReset={...}>
```

⚡ Tricky Point

Error boundaries do **not** catch errors in: event handlers (use try/catch), asynchronous code (setTimeout, promises), server-side rendering, or errors thrown in the boundary itself. There is still **no hook** for error boundaries - it must be a class (or a library wrapping one).

14.7 Suspense and lazy (Recap)

✎ Definition

A recap of `React.lazy` (load a component's code on demand) together with <Suspense> (show a fallback while it loads).

```
<ErrorBoundary fallback={<p>Failed to load</p>}>
  <Suspense fallback={<Spinner />}>
    <LazyReports />
  </Suspense>
</ErrorBoundary>
```

Nested `Suspense` boundaries let different parts of the page reveal independently. With streaming SSR, the server sends HTML for each boundary as it becomes ready.

14.8 Container / Presentational and Custom Hooks

✎ Definition

Separating **container** components (data and logic) from **presentational** components (just UI). Today the logic usually moves into custom hooks instead.

```
function useTxns(bankId) {                  // logic
  return useQuery({ queryKey: ['txns', bankId], queryFn: () => getTxns(bankId) });
}
function TxnTable({ rows }) { /* pure UI */ }       // presentation
function TxnPage({ bankId }) {                       // glue
  const { data = [], isPending } = useTxns(bankId);
  return isPending ? <Spinner /> : <TxnTable rows={data} />;
}
```

14.9 Polymorphic and Slot Components

✎ Definition

A **polymorphic** component can render as different elements through an as prop. **Slot** components accept named pieces of UI as props for flexible layouts.

```
function Button({ as: Component = 'button', variant = 'primary', ...props }) {
  return <Component className={`btn btn-${variant}`} {...props} />;
}
<Button>Save</Button>
<Button as="a" href="/help" variant="link">Help</Button>
<Button as={Link} to="/txns">Transactions</Button>
```

15. Styling, TypeScript and Testing

15.1 Styling Options

> ✎ **Definition**
>
> Ways to style React apps: plain CSS, CSS Modules, utility-first **Tailwind CSS**, CSS-in-JS libraries, and component libraries such as MUI or shadcn/ui.

Approach	Example	Pros	Cons
Plain CSS / global	`import './App.css'`	Simple	Global names collide
CSS Modules	`import s from './Card.module.css'; className={s.card}`	Locally scoped classes, no runtime	Separate files
Tailwind CSS	`className="p-4 rounded-lg bg-white shadow"`	Fast, consistent, tiny CSS output	Long class strings
CSS-in-JS (styled-components, Emotion)	`const Btn = styled.button` with a CSS template	Dynamic styles with props	Runtime cost; issues with Server Components
Component libraries	MUI, Ant Design, Chakra, shadcn/ui (Radix + Tailwind)	Ready-made accessible components	Bundle size, theming effort
Inline style	`style={{ width: pct + '%' }}`	Truly dynamic values	No pseudo-classes or media queries

```
// Conditional classes
<li className={`row ${isActive ? 'active' : ''} ${isFailed ? 'failed' : ''}`} />
import clsx from 'clsx';
<li className={clsx('row', { active: isActive, failed: isFailed })} />
```

15.2 TypeScript with React

✎ Definition

Using **TypeScript** with React to type props, state, events, refs and hooks, which catches bugs at compile time and improves editor autocompletion.

```
// Props
type ButtonProps = React.ComponentProps<'button'> & {     // inherit all native button props
  variant?: 'primary' | 'danger';
  loading?: boolean;
};
export function Button({ variant = 'primary', loading = false, children, ...rest }: ButtonProps) {
  return <button className={`btn-${variant}`} disabled={loading || rest.disabled} {...rest}>{children}</button>;
}

// State
const [user, setUser] = useState<User | null>(null);
const [txns, setTxns] = useState<Txn[]>([]);

// Events
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => setQuery(e.target.value);
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => { e.preventDefault(); };
const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => {};

// Refs
const inputRef = useRef<HTMLInputElement>(null);
// Children and elements
type CardProps = { title: string; children: React.ReactNode; icon?: React.ReactElement };

// Generic component
type ListProps<T> = { items: T[]; render: (item: T) => React.ReactNode; getKey: (item: T) => string };
function List<T>({ items, render, getKey }: ListProps<T>) {
  return <ul>{items.map(i => <li key={getKey(i)}>{render(i)}</li>)}</ul>;
}
```

```
// Reducer actions as a discriminated union
type Action =
  | { type: 'added'; item: Item }
  | { type: 'removed'; id: string }
  | { type: 'cleared' };

// Context
const AuthContext = createContext<AuthValue | null>(null);
```

⚡ Tricky Point

`React.FC` is optional and no longer recommended by many teams: it used to add implicit `children` (removed in React 18 types) and makes generics awkward. Typing the props parameter directly is clearer. `ReactNode` = anything renderable (string, number, null, elements, arrays); `ReactElement` = only JSX elements.

15.3 Testing with React Testing Library

✎ Definition

React Testing Library tests components the way users use them, by querying rendered output by role or text and simulating events, rather than testing implementation details.

React Testing Library (RTL) tests components **the way users use them**: find elements by role/label/text, interact, assert on what is visible - not on internal state.

```tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments the counter', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  expect(screen.getByText(/count: 0/i)).toBeInTheDocument();
  await user.click(screen.getByRole('button', { name: /\+1/ }));
  expect(screen.getByText(/count: 1/i)).toBeInTheDocument();
});

test('shows transactions from the API', async () => {
  vi.spyOn(api, 'getTxns').mockResolvedValue([{ id: 'T1', amount: 500 }]);   // Vitest (jest.spyOn in Jest)
  render(<TxnList />);
  expect(await screen.findByText('T1')).toBeInTheDocument();                 // findBy waits
  expect(screen.queryByText(/loading/i)).not.toBeInTheDocument();
});
```

Query	0 matches	1 match	>1 matches	Async?
`getBy...`	throws	returns	throws	No
`queryBy...`	null	returns	throws	No (use to assert absence)
`findBy...`	rejects	resolves	rejects	Yes (waits up to 1000ms)
`getAllBy / queryAllBy / findAllBy`	throws/[]/rejects	array	array	-

Priority of queries: `getByRole` > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByTestId` (last resort).

> ⚡ **Tricky Point**
>
> The "not wrapped in act(...)" warning means a state update happened after your test finished looking - usually you need to `await` a `findBy` query or `waitFor` for async updates.

Testing custom hooks and other levels

```tsx
import { renderHook, act } from '@testing-library/react';
test('useToggle', () => {
  const { result } = renderHook(() => useToggle());
  act(() => result.current[1]());
  expect(result.current[0]).toBe(true);
});
```

Mock network calls with **MSW (Mock Service Worker)**; end-to-end tests with **Playwright** or **Cypress**.

16. React 18/19, Server Components and Next.js

16.1 React 18 Recap

> ✎ **Definition**
>
> React 18 introduced **concurrent rendering**, automatic batching, transitions, Suspense on the server, streaming SSR and the `createRoot` API.

- **createRoot** and **concurrent rendering**: React can prepare multiple UI versions and interrupt rendering.
- **Automatic batching** everywhere (timeouts, promises, native events).
- **Transitions**: `useTransition`, `startTransition`, `useDeferredValue`.
- **Suspense on the server** + streaming HTML (`renderToPipeableStream`) + selective hydration.
- New hooks: `useId`, `useSyncExternalStore`, `useInsertionEffect`.
- StrictMode double-invokes effects in development.

16.2 React 19 Highlights

✎ Definition

Key React 19 features: Actions and form actions, useActionState, useOptimistic, the use() API, ref as a prop, document metadata support, and stable Server Components.

Feature	What it does
Actions	Async functions in transitions/forms; React manages pending state, errors and optimistic updates
`<form action={fn}>`	Forms submit to functions; FormData passed in; uncontrolled fields reset after success
`useActionState`	`[state, action, isPending]` for an action (was `useFormState` in canaries)
`useFormStatus`	Pending/data of the parent `<form>` from any child (design-system buttons)
`useOptimistic`	Temporary optimistic state during an async action
`use(promise | context)`	Read resources in render; works with Suspense; callable conditionally
ref as a prop	`forwardRef` no longer needed for function components; ref callbacks can return cleanup
`<Context>` as provider	`<ThemeContext value="dark">` instead of `.Provider`
Document metadata	`<title>`, `<meta>`, `<link>` inside components are hoisted to `<head>`
Stylesheet & script support	`<link rel="stylesheet" precedence="...">` with Suspense integration; async scripts deduped
Resource preloading	`preload`, `preinit`, `prefetchDNS`, `preconnect` APIs from react-dom
Better hydration errors	One diff-style message instead of many; tolerates third-party extensions
Custom elements	Full Web Components support
Removed	`propTypes`/`defaultProps` on functions, string refs, legacy context, `ReactDOM.render`, `findDOMNode`, `react-test-renderer/shallow`

Feature	What it does
19.2	`<Activity mode="hidden">` keeps state of hidden UI; `useEffectEvent`; partial pre-rendering; `cacheSignal`

```
// Document metadata in any component (React 19)
function TxnPage({ txn }) {
  return (
    <article>
      <title>{`Txn ${txn.id} | TraceNext`}</title>
      <meta name="description" content={`Details of ${txn.id}`} />
      <h1>{txn.id}</h1>
    </article>
  );
}
// <Activity>: hide a tab without losing its state (19.2)
<Activity mode={tab === 'reports' ? 'visible' : 'hidden'}><Reports /></Activity>
```

16.3 React Server Components (RSC)

✎ Definition

Server Components run only on the server. They can access data directly and send rendered UI to the client with **zero client JavaScript**. Interactive parts are marked `'use client'`.



Server components can render client components, and pass server components to them as children.

Figure 16.1 - Server vs Client Components

Server Components render **on the server only** (at build time or per request). Their code never ships to the browser, they can be `async`, and they can access databases, files and secrets directly. Client Components are the familiar interactive components, marked with the **"use client"** directive at the top of the file.

```
// app/txns/page.tsx  (Server Component by default in Next.js App Router)
import { db } from '@/lib/db';
import { TxnFilters } from './TxnFilters';

export default async function TxnsPage({ searchParams }) {
  const { status } = await searchParams;
  const txns = await db.txn.findMany({ where: { status } });    // direct DB access, no API layer
  return (
    <>
      <TxnFilters />                                                  // client component island
      <table>{txns.map(t => <tr key={t.id}><td>{t.id}</td><td>{t.amount}</td></tr>)}</table>
    </>
  );
}

// app/txns/TxnFilters.tsx
'use client';
import { useRouter } from 'next/navigation';
export function TxnFilters() {
  const router = useRouter();
  return <select onChange={e => router.push(`?status=${e.target.value}`)}>...</select>;
}
```

Server Actions ("use server")

```
// app/actions.ts
'use server';
export async function reverseTxn(formData: FormData) {
  const id = formData.get('id') as string;
  await db.txn.update({ where: { id }, data: { status: 'REVERSED' } });   // runs on server
  revalidatePath('/txns');
}
// in any component
<form action={reverseTxn}><input type="hidden" name="id" value={txn.id} /><button>Reverse</button></form>
```

> ⚡ **Tricky Point**
>
> **"use client" doesn't mean client-only**: Client Components are still pre-rendered to HTML on the server (SSR) and then hydrated. It marks the **boundary** where component code starts being shipped to the browser; everything imported into a client file becomes client code too. Props passed from server to client must be **serializable** (no functions except Server Actions, no class instances).

> ⚠ **Watch Out**
>
> Server Actions are public HTTP endpoints. Always validate input and check authentication/authorisation inside them, exactly like a Web API controller.

Server Component	Client Component
async/await at top level, direct DB/file access	`useState`, `useEffect`, event handlers
Zero JS sent to browser	Code shipped and hydrated
Can import client components	Cannot import server components (but can receive them as `children`)
Secrets stay on server	Only public env vars

16.4 Hydration

> ✎ **Definition**
>
> **Hydration** is React attaching event handlers and state to HTML that was already rendered on the server, making the static page interactive. Mismatches cause hydration errors.

Hydration is React attaching event listeners and state to server-rendered HTML. The first client render must produce **exactly the same output** as the server.

> ⚡ **Tricky Point**
>
> Hydration mismatch causes: `Date.now()`/`Math.random()` in render, `typeof window` branches, locale-dependent formatting, invalid HTML nesting (`<div>` inside `<p>`, `<tr>` without `<tbody>`), browser extensions modifying the DOM. Fix by moving client-only values into `useEffect` or using `suppressHydrationWarning` for unavoidable text like timestamps.

16.5 Next.js App Router Essentials

✎ Definition

Core concepts of **Next.js App Router**: file-based routes in the app/ folder, layouts, Server and Client Components, data fetching and caching, Server Actions, and route handlers.

File (inside app/)	Purpose
`page.tsx`	UI for a route (`app/txns/page.tsx` -> `/txns`)
`layout.tsx`	Shared UI wrapping child routes; preserves state across navigation
`loading.tsx`	Automatic Suspense fallback for the segment
`error.tsx`	Error boundary for the segment (must be a client component)
`not-found.tsx`	404 UI
`route.ts`	API endpoint (GET/POST handlers)
`[id]/`, `[...slug]/`	Dynamic segments
`(group)/`	Route group: organise without affecting the URL
`middleware.ts` / `proxy.ts`	Run code before requests (auth redirects)

Rendering strategies in Next.js: static (default when possible), dynamic per request, streaming with Suspense, ISR (`revalidate`), and Partial Prerendering (static shell + dynamic holes).

16.6 Accessibility (a11y) Essentials

✎ Definition

Building UIs that everyone can use, including keyboard and screen-reader users: semantic HTML, labels, alt text, focus management, ARIA only when needed, and sufficient colour contrast.

- Use semantic HTML: `<button>` for actions (not clickable `<div>`), `<a>` for navigation, `<label htmlFor>` for inputs, headings in order.
- Every image needs `alt` (empty `alt=""` for decorative images).
- Make custom widgets keyboard accessible (Tab, Enter, Space, Escape) with correct `role` and `aria-*` attributes - or use a headless library (Radix, React Aria, Headless UI).
- Manage focus when opening/closing modals; announce dynamic changes with `aria-live`.
- Test with keyboard only, a screen reader and tools like axe / eslint-plugin-jsx-a11y.

16.7 Security Checklist

✎ Definition

Key front-end security practices: avoid unsanitised HTML (XSS), validate URLs, keep secrets out of client bundles, enforce authorisation on the server, and use secure cookies.

- Rely on React's automatic escaping; sanitize anything passed to `dangerouslySetInnerHTML` (DOMPurify).
- Validate URLs before using them in `href`/`src` (block `javascript:` URLs).
- No secrets in client bundles; everything in `VITE_`/`NEXT_PUBLIC_` is public.

- Authorisation must be enforced on the server (API, Server Actions), never only in route guards.
- Prefer HttpOnly, Secure, SameSite cookies for session tokens; protect against CSRF for cookie-based auth.
- Keep dependencies updated (`npm audit`) - React Server Components had critical security fixes in 2025, so stay on patched versions.

17. Tricky Interview Questions and Output Puzzles

Predict the output before reading the answer. These are among the most frequently asked React interview scenarios.

17.1 Output Prediction Puzzles

> ✎ **Definition**
>
> Short React snippets where you **predict the exact output or behaviour**, testing your understanding of state snapshots, batching, closures, keys and effects.

Puzzle 1 - Multiple setState calls

```
function App() {
  const [n, setN] = useState(0);
  const click = () => { setN(n + 1); setN(n + 1); setN(n + 1); };
  return <button onClick={click}>{n}</button>;
}
// Clicked once. What is displayed?
```

> ⚡ **Tricky Point**
>
> Answer: **1**. Each call uses the same snapshot `n = 0`. With `setN(p => p + 1)` three times it would be **3**.

Puzzle 2 - Mixing value and updater

```
const click = () => {
  setN(n + 5);          // n = 0
  setN(p => p + 1);
  setN(42);
  setN(p => p * 2);
};
```

> ⚡ **Tricky Point**
>
> Answer: **84**. React processes the queue in order: 0+5 = 5, then 5+1 = 6, then replace with 42, then 42*2 = 84.

Puzzle 3 - Reading state after setting it

```
const click = () => {
  setN(n + 1);
  console.log(n);
  setTimeout(() => console.log(n), 3000);
};
// n is 0. Click once.
```

> ⚡ **Tricky Point**
>
> Answer: logs **0** and, after 3 seconds, **0** again. The timeout closure captured `n` from the render in which it was created. State is a snapshot per render.

Puzzle 4 - Mutating an array in state

```
const [items, setItems] = useState(['a']);
const add = () => { items.push('b'); setItems(items); };
```

> ⚡ **Tricky Point**
>
> Answer: **no re-render** (UI still shows "a"). Same array reference, so `Object.is(prev, next)` is true and React bails out. Use `setItems([...items, 'b'])`.

Puzzle 5 - The 0 rendering bug

```
const items = [];
return <div>{items.length && <List items={items} />}</div>;
```

> ⚡ **Tricky Point**
>
> Answer: renders **0**. `0 && ...` returns 0, and React renders numbers. Use `items.length > 0 &&`.

Puzzle 6 - Effect order

```
function App() {
  console.log('render');
  useEffect(() => { console.log('effect'); return () => console.log('cleanup'); });
  useLayoutEffect(() => { console.log('layout'); });
  return null;
}
// First mount in production, then one re-render.
```

> ⚡ **Tricky Point**
>
> Answer: mount: **render, layout, effect**. Re-render: **render, layout, cleanup, effect** (the previous effect's cleanup runs right before the new effect). In development with StrictMode you'd see extra render/effect/cleanup calls.

Puzzle 7 - Stale interval

```
function Timer() {
  const [s, setS] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setS(s + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <p>{s}</p>;
}
```

> ⚡ **Tricky Point**
>
> Answer: shows **1** forever. The interval callback closes over `s = 0` from the first render, so it keeps setting 0 + 1. Fix: `setS(prev => prev + 1)` or add `s` to deps.

Puzzle 8 - Parent and child render order

```
function Child() { console.log('Child render'); useEffect(() => console.log('Child effect')); return null; }
function Parent() { console.log('Parent render'); useEffect(() => console.log('Parent effect')); return <Child />; }
```

⚡ Tricky Point

Answer: **Parent render, Child render, Child effect, Parent effect**. Rendering goes top-down; effects run bottom-up (children first, like componentDidMount).

Puzzle 9 - memo with inline object

```
const Child = memo(({ config }) => { console.log('child'); return null; });
function Parent() {
  const [n, setN] = useState(0);
  return <><button onClick={() => setN(n + 1)}>+</button><Child config={{ dark: true }} /></>;
}
// Click 3 times.
```

⚡ Tricky Point

Answer: "child" logs **4 times** (mount + 3). `{ dark: true }` is a new object each render, so memo's shallow comparison fails. Fix with `useMemo` or define the object outside the component.

Puzzle 10 - Setting the same state

```
const [v, setV] = useState('a');
console.log('render');
<button onClick={() => setV('a')}>Same</button>
```

⚡ Tricky Point

Answer: clicking logs **nothing** (React bails out because the value is identical). React may occasionally render the component once more before bailing out, but it won't render children or commit.

Puzzle 11 - Index as key

```
const [list, setList] = useState(['Ali', 'Sam']);
{list.map((name, i) => <li key={i}>{name} <input /></li>)}
// Type "x" in Ali's input, then remove 'Ali' from the array.
```

⚡ Tricky Point

Answer: "Sam" now appears next to the input containing **"x"**. React matched the remaining item to key 0 and reused the first `<li>` (and its uncontrolled input DOM state). Use stable keys (`key={name}` or an id).

Puzzle 12 - Conditional hook

```
function Profile({ show }) {
  if (show) { const [x] = useState(1); }
  const [y, setY] = useState(2);
}
```

> ⚡ **Tricky Point**
>
> Answer: when `show` flips, React throws "Rendered more/fewer hooks than during the previous render" (or mixes up state). Hooks are matched by call order. The linter flags this as an error.

Puzzle 13 - Context default value

```
const Ctx = createContext('default');
function Show() { return <p>{useContext(Ctx)}</p>; }
<Ctx.Provider value={undefined}><Show /></Ctx.Provider>
```

> ⚡ **Tricky Point**
>
> Answer: renders **nothing** (`undefined`), not "default". The default is used only when there is **no provider** above the consumer.

Puzzle 14 - Event handler invocation

```
<button onClick={alert('Hi')}>Greet</button>
```

> ⚡ **Tricky Point**
>
> Answer: the alert pops up **during render** (every render), and clicking does nothing, because `alert('Hi')` is called immediately and its return value `undefined` becomes the handler. Use `onClick={() => alert('Hi')}`.

17.2 Frequently Asked Conceptual Questions

✎ Definition

The React theory questions interviewers ask most often, each with a concise answer you can give in one or two sentences.

Question	Crisp answer
What is React?	A JavaScript library for building UIs from components using a declarative, state-driven model.
What is JSX?	Syntax sugar compiled to `jsx()`/`createElement` calls that return plain element objects.
Virtual DOM?	In-memory tree of elements; React diffs new vs old and applies minimal DOM changes.
Reconciliation?	The diffing process: different types replace subtrees, same types update in place, keys match lists.
What is Fiber?	React 16+ reconciler: interruptible, prioritised work units enabling concurrent rendering.
Props vs state?	Props are read-only inputs from the parent; state is the component's own changing memory.
Why keys?	Stable identity for list items so React preserves state and updates efficiently.
Controlled vs uncontrolled?	Controlled: value in React state via value + onChange. Uncontrolled: DOM holds value, read via ref.
Why immutability?	React detects changes by reference (`Object.is`); mutations are invisible and break memo/history.
useEffect vs useLayoutEffect?	useEffect runs after paint (non-blocking); useLayoutEffect runs before paint for DOM measurements.

Question	Crisp answer
useMemo vs useCallback?	Cache a value vs cache a function; both re-compute when dependencies change.
React.memo?	Skips re-rendering if props are shallowly equal.
Context vs Redux?	Context transports values without selectors (all consumers re-render); Redux is a store with selectors, middleware, DevTools.
What are hooks? Rules?	Functions to use React features in function components. Call at top level, only from React functions.
What is prop drilling? Fixes?	Passing props through unrelated layers; fix with composition, context or a store.
What is lifting state up?	Moving shared state to the closest common parent and passing it down.
Error boundary?	Class component that catches render errors in its subtree and shows a fallback.
What is a portal?	Rendering children into a different DOM node while staying in the React tree.
SSR vs CSR vs SSG?	HTML built per request on the server / in the browser / at build time.
What is hydration?	Attaching React to server-rendered HTML to make it interactive.
Server vs Client Components?	Server: run only on server, no JS shipped, can be async. Client: interactive, "use client".
What does StrictMode do?	Dev-only double rendering/effects and deprecation warnings to surface bugs.
Synthetic events?	Cross-browser wrappers around native events, delegated at the root container.
What is batching?	Grouping multiple state updates into one re-render (automatic everywhere since React 18).
useTransition?	Marks an update as non-urgent so React can keep urgent updates (typing) responsive.
Code splitting?	Loading parts of the bundle on demand with `import()`, `lazy` and `Suspense`.
Custom hook?	A `use*` function reusing stateful logic; each call has independent state.
Pure component?	Same inputs produce the same output with no side effects during render.
Why not mutate props?	Props belong to the parent; mutating breaks one-way data flow and change detection.
Class lifecycle to hooks?	didMount/didUpdate/willUnmount map to useEffect with deps and cleanup.

17.3 Machine-Coding Questions to Practise

> ✎ **Definition**
>
> **Machine-coding** rounds ask you to build a small working UI feature, such as a todo list, autocomplete or pagination, within a time limit. This section lists common ones to practise.

```
// 1) Counter with step, min/max and reset
// 2) Todo list: add, toggle, delete, filter (All/Active/Done), persist to localStorage
// 3) Debounced search with API call, loading state and race-condition handling
// 4) Paginated table with sorting and search params in the URL
// 5) Infinite scroll with IntersectionObserver
// 6) Modal with portal, focus trap and Escape-to-close
// 7) Star rating component (controlled + uncontrolled)
// 8) Accordion / Tabs as compound components
// 9) Countdown timer with pause/resume (useRef for interval id)
// 10) Form with validation: email, password strength, confirm password
// 11) Nested comments (recursive component)
// 12) Traffic light / stepper using useReducer
```

Example: infinite scroll

```
function useInfiniteScroll(onLoadMore, hasMore) {
  const sentinelRef = useRef(null);
  useEffect(() => {
    if (!hasMore) return;
    const observer = new IntersectionObserver(([entry]) => { if (entry.isIntersecting) onLoadMore(); });
    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [onLoadMore, hasMore]);
  return sentinelRef;
}
function Feed() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const loadMore = useCallback(async () => {
    const next = await api.getPage(page);
    setItems(prev => [...prev, ...next]);
    setHasMore(next.length > 0);
    setPage(p => p + 1);
  }, [page]);
  const sentinel = useInfiniteScroll(loadMore, hasMore);
  return <>{items.map(i => <Card key={i.id} {...i} />)}<div ref={sentinel} /></>;
}
```

Example: recursive nested comments

```
function Comment({ comment }) {
  const [open, setOpen] = useState(true);
  return (
    <li>
      <p>{comment.text} {comment.replies?.length > 0 && <button onClick={() => setOpen(o => !o)}>{open ? '-' : '+'}</button>}</p>
      {open && comment.replies?.length > 0 && (
        <ul>{comment.replies.map(r => <Comment key={r.id} comment={r} />)}</ul>
      )}
    </li>
  );
}
```

18. Quick Revision Cheat Sheet

18.1 Syntax at a Glance

✎ Definition

A compact reference of everyday React syntax (components, props, state, effects, hooks, context, routing) in one place for quick lookup.

```
// Component + props
function Card({ title, children, onClose = () => {} }) { return <div>{title}{children}</div>; }
// State
const [v, setV] = useState(init);   setV(x);   setV(prev => next);
const [state, dispatch] = useReducer(reducer, initialState);
// Effects
useEffect(() => { /* setup */ return () => { /* cleanup */ }; }, [deps]);
// Refs
const ref = useRef(null);   <input ref={ref} />   ref.current.focus();
// Context
const Ctx = createContext(null);   <Ctx.Provider value={v}>...</Ctx.Provider>   useContext(Ctx);
// Memoization
const val = useMemo(() => compute(a), [a]);   const fn = useCallback(x => do(x), []);   memo(Comp);
// Lists & conditions
{items.map(i => <Row key={i.id} {...i} />)}   {cond ? <A /> : <B />}   {n > 0 && <C />}
// Events & forms
<input value={v} onChange={e => setV(e.target.value)} />   <form onSubmit={e => { e.preventDefault(); }}>
// Immutable updates
setObj(o => ({ ...o, key: val }));   setArr(a => [...a, x]);   a.filter(...)   a.map(...)
// Lazy + Suspense
const Page = lazy(() => import('./Page'));   <Suspense fallback={<Spin />}><Page /></Suspense>
// Router
<Route path="/txns/:id" element={<Txn />} />   useParams()   useNavigate()   <Link to="/">
// React 19
const [state, action, pending] = useActionState(fn, init);   <form action={action}>   use(promise)
```

18.2 Top 25 Tricky Points

✎ Definition

The 25 most important React "gotchas" from this guide, gathered for fast revision before an interview.

- State is a **snapshot**: reading it right after `setX` gives the old value.
- Use the **updater function** when the next state depends on the previous one.
- Never **mutate** state or props; create new objects/arrays (spread copies are shallow).
- `{count && <X />}` renders **0** when count is 0.
- Keys must be **stable and unique among siblings**; index keys break reorderable lists; random keys remount everything.
- Changing a component's **key resets** its state.
- Don't define components **inside** components.
- `onClick={fn()}` calls immediately; use `onClick={fn}` or an arrow.
- `<button>` inside a form defaults to **type="submit"**.
- Initial state is used **only once**; `useState(prop)` doesn't sync with later prop changes.
- Effects run **after paint**; `useLayoutEffect` before paint.
- Effects run **twice** in development StrictMode - write proper cleanup.
- Objects/functions in dependency arrays are **new every render**.

- Missing dependencies cause **stale closures**; respect `exhaustive-deps`.
- Fetch in effects needs **abort/ignore** cleanup to avoid race conditions.
- Don't use effects for derived data or for reacting to user events.
- Hooks must be called **in the same order**: no conditions, loops or early returns before them.
- `memo` fails with inline objects/functions/children unless memoized.
- **All context consumers re-render** on value change; memoize and split contexts.
- Context default value applies only **without a provider**.

- `fetch` doesn't reject on 404/500 - check `res.ok`.
- Error boundaries don't catch **event handler or async** errors.
- Portal events bubble through the **React tree**, not the DOM tree.
- Client-side route guards are **not security**; env vars in the bundle are **public**.
- Hydration mismatches come from **non-deterministic render** output (dates, random, window checks).

18.3 Suggested Study Roadmap

✎ Definition

A suggested order of study, from JavaScript foundations to advanced React and Next.js, with practice projects for each stage.

Week	Focus	Practice
1	Chapters 1-4: JS essentials, JSX, components, props	Build static pages from a design: cards, tables, layouts
2	Chapters 5-7: state, events, forms, rendering	Todo app, controlled forms with validation, puzzles 1-5
3	Chapters 8-9: effects and all hooks, custom hooks	Debounced search with cancellation; write 5 custom hooks
4	Chapters 10-13: context, Redux Toolkit, Router, React Query, performance	Mini transaction dashboard: routing, filters in URL, React Query, virtualized table
5	Chapters 14-16: patterns, TypeScript, testing, React 19, Next.js	Convert the dashboard to TypeScript, add RTL tests, try a Next.js server component page
6	Chapter 17: puzzles + machine coding	Timed machine-coding rounds (45-60 min each); explain each puzzle aloud

i Note

Keep the React DevTools open while practising. Watching which components highlight on each render teaches you more about re-rendering than any article.